

Behavior in higher-order languages

Admissible observers, relative pushouts, and the encoding of extended theories

C.B. Wells, Michael Stay, L.G. Meredith

f1r3fly.io

Second draft

Abstract

Every construction that derives a labelled transition system from a rewrite relation assumes that all contexts may observe. That assumption fails in exactly the situations one wants the construction for. We therefore make the class of admissible observers a *parameter*: where Leifer and Milner [2000] restrict which contexts may *react*, we additionally restrict which contexts may *observe*, and compute relative pushouts inside that class rather than in the ambient theory. The resulting $\mathcal{D}$ -relative bisimilarity is a congruence for $\mathcal{D}$, which is the statement one actually wants, and $\mathcal{D}$ itself is not chosen by hand: it is determined by an equivalence at the ground types.

The evidence that this is the right parameterization is that three apparently unrelated facts become one theorem instantiated three times — the failure of congruence under reflection in the rho calculus, the requirement that an encoding be probed only by encoded contexts, and the correctness of compiling away built-in data types. A companion paper [Meredith, 2026a] proves a full-abstraction theorem for an encoding of the π calculus in the rho calculus in which the restriction to encoded observers is not a convenience but is shown to be forced; we take that encoding as our running instance.

Contents

1	Introduction	3
1.1	One instance, in full, before any machinery	3
1.2	The obstacle	3
1.3	The move	4
1.4	Contributions	4
1.5	Status of the results	5
1.6	What is deliberately absent	6
2	Lambda theories	6
2.1	Meta-contexts and the rewrite proposition	7
2.2	What is in the appendices, and a reading path	9
3	The theories we treat	9
4	Admissible observers	11
4.1	Observation relations	11
4.2	The circularity, and why it is respectable	12
4.3	The rho calculus: names are discrete	13
4.4	Encodings: the observation relation transported	13

5	Relative pushouts, relatively	14
5.1	Relative pushouts	14
5.2	Two things Leifer and Milner did not need	14
5.3	Which contexts are observations	15
5.4	The derived transition system	15
5.5	Normality and substitution	16
5.6	Congruence	17
5.7	Existence of relative IPOs	17
5.8	Enumerating redex IPOs	18
5.9	Working up to bisimilarity	19
6	First instance: reflection in the rho calculus	20
7	Second instance: encodings	21
7.1	What an encoding must preserve	21
7.2	Eliminating replication	22
7.3	π into rho	23
7.4	Relative minimality, and what transfers	26
7.5	What remains: a comparison, not a construction	26
7.6	Separation	27
8	Third instance: extension by data types	28
8.1	Extensions and models	28
8.2	Canonicity	28
8.3	The name discipline	29
8.4	Booleans, in full	30
8.5	The general theorem	31
8.6	What is out of scope, and why	31
9	Related work	32
10	A proposal: morphisms should carry contexts	32
10.1	The received definition is incomplete	32
10.2	The proposal	33
10.3	Four things to settle first	34
10.4	What this asks of the companion documents	34
11	Conclusion and open problems	35
A	Inference rules for lambda theories	40
B	Free extension of theories	41
C	Meta-theories	42
D	String diagrams	42

1 Introduction

F1R3FLY is a concurrent computing platform based on the reflective higher-order π -calculus [Meredith and Radestock, 2005]. Its language-definition layer, MeTTaIL, lets a user present a domain-specific language as a signature with equations and rewrite rules, and compiles that presentation into the rho calculus for execution. For this to be more than a syntactic convenience, the user needs a guarantee: that reasoning carried out in the language they defined remains valid in the calculus their program is actually run in.

This paper is about the shape of that guarantee, and about a single obstacle that recurs whenever one tries to state it.

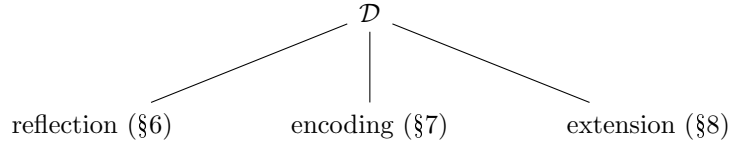
1.1 One instance, in full, before any machinery

A reader who wants to know whether the apparatus below is worth its cost should have a case in hand first. Here is one, and it is not a toy.

The rho calculus has no name restriction and no supply of atoms: names are quoted processes. Encoding the π calculus into it therefore means implementing ν . Meredith [2026a] gives such an encoding, in which each syntactic node derives two child addresses from its own by communication, and proves it fully abstract: two π processes are bisimilar exactly when their images are. The equivalence on the target is context-labelled, and the contexts allowed as labels are the images $\llbracket c \rrbracket$ of π contexts.

That last clause is not a hedge. Take P and $P|0$, which are structurally congruent in π . Their images differ: $\llbracket P|0 \rrbracket$ carries the administrative traffic of a parallel node, and so has a barb on the address at which that node was invoked, while $\llbracket P \rrbracket$ has none. The address is computable from the free names of P , so *some* rho context can name it and separate the two images. No encoded context can, because no π context mentions it. A sharper version of the same phenomenon uses reflection rather than scaffolding: an arbitrary rho context may drop a name it receives, running the process the name quotes, and so read a ν -generated address as structure. Lybech [2022a] shows that this is not a defect of the encoding but a fact about the calculi: rho manufactures new free — hence observable — names at runtime, and no encoding of π into rho can be fully abstract against unrestricted rho observers.

So the theorem cannot be stated without naming a class of contexts, and the class that works is the image of the source. That is the whole content of this paper, in one example. What remains is to notice that the same thing happens twice more, that the class is determined rather than stipulated, and that once labels are contexts the class is not a parameter of the theorem but a component of the translation.



In the left case $\mathcal{D}$ excludes the contexts with a hole under $@$; in the middle it is the image $\llbracket \mathcal{D} \rrbracket$ of the source contexts; in the right it is the image of the extended theory's contexts. One definition, three choices, and in each the good theorem is false without the restriction and true with it.

1.2 The obstacle

Behavioral equivalence is always relative to a class of observers. A labelled transition system fixes that class by fixing the labels, and since Milner [1982] the labels have been supplied by hand. Leifer and Milner [2000] showed that they need not be: labels can be *derived* as the minimal contexts enabling a rewrite, minimality being a universal property (a relative pushout), and bisimilarity for the derived system is then automatically a congruence.

The difficulty is that “minimal context” presumes we know which contexts are eligible. Leifer and Milner already need one restriction here — a subcategory $\mathcal{D}$ of *reactive* contexts, those in which a rewrite may occur — but they take the eligible *observers* to be all of the ambient category. In every situation we care about, that is too many.

- **Reflection.** In the rho calculus, $@$ turns a process into a name and $*$ turns it back. Communication tests names for *equality*, not for bisimilarity. So if $p \approx q$ with $p \neq q$, the context $\text{out}(@(-), z)$ separates them: bisimilarity is not a congruence for contexts with a hole under $@$. It is a congruence for the rest.
- **Encodings.** An encoding $\llbracket - \rrbracket : S \rightarrow T$ lands its image in a much larger theory. Arbitrary T -contexts can read the encoding’s plumbing — name supplies, servers, dead branches — as observable structure, separating the images of source-equivalent programs. Only the images $\llbracket c \rrbracket$ of source contexts respect the intended abstraction.
- **Extension.** A theory with a built-in `Bool` can be compiled into one without, but the compiled conditional leaves an unreachable branch behind. It is unreachable only because no *encoded* context ever addresses it.

These are the same problem. In each case there is a class $\mathcal{D}$ of contexts for which the theory is well-behaved, a larger class for which it is not, and no way to state the good theorem without naming $\mathcal{D}$.

1.3 The move

We therefore take $\mathcal{D}$ as data, and compute relative pushouts inside it.

Given a lambda theory T and a subcategory $\mathcal{D}$ of admissible observers, a $\mathcal{D}$ -relative IPO is a context in $\mathcal{D}$ that is minimal *among contexts in $\mathcal{D}$* enabling a given rewrite — not minimal in T . These differ: the T -minimal enabler of a transition out of $\llbracket p \rrbracket$ is typically strictly smaller than any $\llbracket c \rrbracket$, because $\llbracket c \rrbracket$ carries plumbing that minimality in T strips away. Insisting on T -minimality thus forces observations outside the image; insisting on encoded contexts abandons minimality. Relative minimality keeps both.

We do not choose $\mathcal{D}$ arbitrarily. Section 4 shows that $\mathcal{D}$ is determined by a much simpler datum: an *observation relation*, a family $\approx_A$ of equivalences indexed by the types of the theory, built up from a choice at the ground types by the usual logical-relation clauses. An operation is admissible exactly when it preserves the relation. The rho calculus story then falls out of a single decision — that $\approx_{Nm}$ is equality — and the encoding story falls out of another — that the observation relation on the target is the image of the one on the source.

1.4 Contributions

1. A presentation of programming-language specifications as **lambda theories**: categories with finite limits whose subobject fibration is cartesian closed, carrying a distinguished type Pr and rewriting as a *proposition* $(\rightsquigarrow) \multimap Pr \times Pr$. Types, terms, equations and rewrites live in one structure, and currying entailments gives rewriting under binders (§2).
2. An **open** form of the Leifer–Milner construction. Their reactive systems fix a distinguished object 0 as the domain of IPOs, restricting attention to ground terms; their proofs do not use it. Dropping it yields a transition system on open terms (§5).
3. **Observation relations and admissible observers** (§4): the class $\mathcal{D}$ derived from a logical relation on types, with admissibility as preservation.

4. **$\mathcal{D}$ -relative pushouts**, the derived $\mathcal{D}$ -relative transition system, and the $\mathcal{D}$ -relative congruence theorem: $\sim_{\mathcal{D}}$ is a congruence for $\mathcal{D}$ (§5). We also give the construction in $\mathcal{D}/\approx$, where labels are compared coinductively rather than syntactically, which is what lets equations of a source theory be satisfied only up to bisimilarity in a target.
5. Three instantiations: reflection in the rho calculus (§6), encodings (§7), and extension by built-in data types (§8), where we prove that the equations of a finitary algebraic theory become weak bisimulations under a Scott-style encoding, worked in full for `Bool`.
6. A proposal (§10) that morphisms of graph-structured lambda theories carry a map on contexts as well as on terms. The observer subcategory is then not a parameter but the image of that map; the notion of encoding used here becomes the notion of morphism; and hosting and exhausting become faithfulness and density of a functor.
7. A **worked flagship**: the encoding of π into rho of Meredith [2026a], whose full-abstraction theorem is stated for exactly the observer class this paper constructs, and whose necessity proposition shows the class cannot be enlarged (§7.3). We state the one lemma that transfers that theorem to Definition 37: that relative minimality loses no separating power in the image (§7.4), of which the direction we need for soundness is proved here and the converse is Obligation 50.
8. A reading of the encodability and separation results of Lybech [2022a] for the rho calculus in these terms (§7.3, §7.6). His $\mathcal{N}$ -restricted barbed bisimilarity is $\mathcal{D}$ -relative bisimilarity with $\mathcal{D}$ stipulated as a set of observable names rather than derived; and his separation theorem turns on reflection manufacturing observable free names at runtime, which is the same fact as the inadmissibility of @.

1.5 Status of the results

This is a working draft, and we think it more useful to say plainly where each result stands than to present a uniform surface. `PROVED` means a proof is given here or is immediate from a cited result; `SKETCH` means the argument is given but a case analysis is incomplete; `OBLIGATION` means the statement is what we intend to prove and the proof is outstanding; `PROPOSAL` marks the definitional proposal of §10, whose content is a choice about the category rather than a claim about a fixed one.

Result	Where	Status
Observation relation determines $\mathcal{D}$	Prop. 7	PROVED
@ is not admissible	Prop. 10	PROVED
$\mathcal{D}$ -relative congruence (strong)	Thm. 23	PROVED
Existence of $\mathcal{D}$ -relative IPOs	Prop. 25	SKETCH
Decomposition-closure of $\llbracket C_\pi \rrbracket$ in rho	Ob. 26	OBLIGATION
$\mathcal{D}$ -relative congruence (weak)	Thm. 24	OBLIGATION
Congruence for rho, admissible contexts	Thm. 34	PROVED
Redex IPOs exist for rho and π	Thm. 28	SKETCH
Replication elimination $\text{rho}^1 \rightarrow \text{rho}$	Thm. 39	PROVED
$\pi \rightarrow \text{rho}$, against Gorla-style criteria	Thm. 42	PROVED [Lybech, 2022a]
$\pi \rightarrow \text{rho}$, fully abstract for $\llbracket C_\pi \rrbracket$	Thm. 43	PROVED [Meredith, 2026a]
The restriction to $\llbracket C_\pi \rrbracket$ is forced	Prop. 44	PROVED [Meredith, 2026a]
Soundness descends to relative-IPO labels	Prop. 49	PROVED
Relative minimality separates as well	Ob. 50	OBLIGATION
Those criteria imply Definition 37	Ob. 52	OBLIGATION
Separation: rho does not embed in π	Thm. 55	PROVED [Lybech, 2022a]
The same, derived $\mathcal{D}$ -relatively	Ob. 56	OBLIGATION
Extension equations become weak bisimulations	Thm. 67	SKETCH
The same, for Bool	Prop. 65	PROVED
Retraction $T \otimes A \rightarrow T$ uniform in T	Ob. 68	OBLIGATION
$\mathcal{D}$ is the image of a morphism's context map	Prop. 74	PROPOSAL
Encodings are morphisms of theories	Prop. 75	PROPOSAL
Generated logics are functorial	Conj. 77	OBLIGATION

1.6 What is deliberately absent

Earlier drafts of this material attempted the ambient calculus, the λ -calculus, name-passing λ , combinators, and a general theory of “weakened” theories with relative pushouts taken over an internal congruence. All are omitted.

The ambient calculus is omitted because it fails our hypotheses for a reason worth stating: the operation $n[-]$ does not commute with parallel composition, $n[p|q] \neq n[p]|q$, so the redex-IPO enumeration of §5 does not apply. Bonchi et al. [2009] show that the IPO semantics for ambients is in any case too fine and must be coarsened by barbs; the correct treatment is a separate paper. The λ -calculus is omitted because it has no parallel composition at all, and because Di Gianantonio et al. [2009] already treat it with second-order contexts. The internal-congruence construction is subsumed: §5.9 obtains the same effect by taking relative pushouts in $\mathcal{D}/\approx$, which needs no new framework.

2 Lambda theories

Algebraic structures, such as rings or automata, are defined by operations that satisfy equations. Such an operation has many inputs and one output, and thus can be described as a morphism $f: \prod A_i \rightarrow B$ from a product of types to a type. Hence *cartesian categories* have become the

general setting for universal algebra [Adámek et al., 2010]. We denote such an operation as follows.

$$a_1 : A_1, \dots, a_n : A_n \vdash f : B$$

Yet to model programming languages, a more expressive framework is needed, as a program may be an input to another program: a higher-order operation, whose inputs can be other operations. This is a morphism in a *closed* cartesian category, with both product types and *function types*, whose terms are formed by lambda abstraction and used in beta reduction [Lambek and Scott, 1988].

$$\frac{\Gamma, x : X \vdash c : A}{\Gamma \vdash \lambda x. c : [X \rightarrow A]} \quad \frac{\Gamma \vdash t : X \quad \Gamma \vdash f : [X \rightarrow A]}{\Gamma \vdash f(t) : A}$$

To save space, we may write $[X \rightarrow Y]$ as $[X \backslash Y]$ and $X \times Y$ as X, Y .

For example in the π calculus [Milner, 1993], an input process hosts a function from names to processes, which can evaluate when in parallel with an output process on the same channel.

$$n : Nm, \lambda x. c : [Nm \backslash Pr] \vdash \text{in}(n, \lambda x. c) : Pr \quad \Gamma \vdash \text{out}(n, a) \mid \text{in}(n, \lambda x. c) \rightsquigarrow c(a)$$

The symbol $\rightsquigarrow$ denotes a *rewrite* from one program to another, as in a basic step of computation. This is a binary relation on programs, found in virtually every language specification, and presented in the style of Plotkin [2004]. Reasoning about programs involves a combination of types, operations, equations, and rewrites; yet most literature does not explicate their universe of discourse as one cohesive logical structure.

$$p : Pr, q : Pr \vdash (p \rightsquigarrow q) \text{ prop}$$

Categorical logic provides a canonical setting: the structure of a *logic over a type system* [Jacobs, 1999]. Rewriting is a *proposition* on pairs of programs, and dynamics is expressed by *entailments* thereof. As each predicate depends on a context of types, the category (poset) of predicates and entailments depends on the category of types and terms: together they form a **fibred category**.

$$\begin{array}{ccc} \text{SubT} & \varphi & \vdash \quad \psi \\ \downarrow & & \\ \text{T} & S \xrightarrow{\quad f \quad} & T \end{array}$$

The morphism above expresses that for any $s : S$, $\varphi[s]$ entails $\psi[f(s)]$, written $s : S \mid \varphi \vdash \psi[f]$.

Every category with finite limits T determines a fibred category of subobjects $\text{SubT} \rightarrow \text{T}$, in which an object over Γ is a subobject $\varphi \rightarrow \Gamma$, entailment is inclusion, and substitution is pullback. The internal language of this structure is equational logic; the inference rules are collected in Appendix A, together with the characterization theorem of Jacobs [1999] that licenses their use.

2.1 Meta-contexts and the rewrite proposition

We will define rewriting in terms of a distinguished type Pr and a proposition $(\rightsquigarrow) \rightarrow Pr \times Pr$. To define operations on rewrites, we introduce the following concepts.

Definition 1. Consider the category of endofunctors $\text{Cat}(\text{T}, \text{T})$. Define $\text{C}[\text{T}]$ to be the subcategory generated by $- \times X$ and $[X \rightarrow -]$ over all $X : \text{T}$. We call such a composite a **meta-context**, and denote one by $\langle - \rangle : \text{T} \rightarrow \text{T}$, or $\langle - \rangle_i$ when there are multiple. These extend to fibred endofunctors $- \wedge \top_X$ and $\top_X \Rightarrow -$, the composites of which form $\langle - \rangle : \text{SubT} \rightarrow \text{SubT}$ for propositions.

Since $A \cong 1 \times A$, the identity is a meta-context. Meta-contexts are equipped with the following isomorphisms.

$$\begin{aligned}
A \times (B \times -) &\cong (A \times B) \times - \\
A \rightarrow (B \rightarrow -) &\cong (A \times B) \rightarrow - \\
A \rightarrow (B \times -) &\cong (A \rightarrow B) \times (A \rightarrow -)
\end{aligned}$$

So in general, $\langle - \rangle = A \times [B \rightarrow -]$ for some $A, B : \mathbb{T}$. To define equations with binding operations, we need the monoidal strength st_B^A of each $[B \rightarrow -]$ with respect to products of $\mathbb{T}$ [Kock, 1972]:

$$\begin{array}{ccc}
& [B \rightarrow A] \times [B \rightarrow -] & \\
\eta_B^A \nearrow & & \searrow \cong \\
A \times [B \rightarrow -] & \xrightarrow{st_B^A} & [B \rightarrow (A \times -)]
\end{array}$$

where $\eta_B^A : A \rightarrow [B \rightarrow A]$ is the currying of the projection $\pi_A : A \times B \rightarrow A$. This converts a term a with b not occurring free in a into a term $\lambda b.a$ constant at a .

Definition 2. An *equational logic with binding and rewriting* is a category with finite limits such that $\pi : \text{SubT} \rightarrow \mathbb{T}$ is cartesian closed, with a type Pr and a proposition $(\rightsquigarrow) \mapsto Pr \times Pr$ with entailments: base rewrites $\rightsquigarrow_B = \{\Gamma \mid \top \vdash p \rightsquigarrow q\}$, and rewrite operations $\rightsquigarrow_O$ of the form below.

$$\prod \langle \Gamma_i \rangle_i \mid \bigwedge \langle p_i \rightsquigarrow q_i \rangle_i \vdash f(\vec{p}) \rightsquigarrow f(\vec{q})$$

We shorten to the name **lambda theory**, as in the classic example of the λ -calculus.

Definition 3. A *presentation* of a lambda theory $\mathbb{T}$ consists of:

- a set of **base types** $\mathbb{T}_{\text{type}} = \{A\}$, including Pr
- a set of **operations** $\mathbb{T}_{\text{oper}} = \{\Gamma \vdash t : A\}$, with $A \in \mathbb{T}_{\text{type}}$
- a set of **propositions** $\mathbb{T}_{\text{prop}} = \{\varphi\}$, including $(p : Pr, q : Pr \vdash p \rightsquigarrow q)$
- a set of **entailments** $\mathbb{T}_{\text{ent}} = \{\Gamma \mid \Theta \vdash \varphi\}$, including $\rightsquigarrow_B$ and $\rightsquigarrow_O$ above.

A theory is **finitely presented** if all four sets are finite.

The source l of a rewrite $l \rightsquigarrow r$ is called a **redex**. Rewrite operations are composed as follows, forming the set of **rewrite constructors** $\rightsquigarrow_C$.

$$\frac{f : (\rightsquigarrow_O)(\prod \langle Pr \rangle_i)}{f : (\rightsquigarrow_C)(\prod \langle Pr \rangle_i)} \quad \frac{\{c_i : (\rightsquigarrow_C)(\Gamma_i)\} \quad f : (\rightsquigarrow_O)(\prod \langle Pr \rangle_i)}{f(\prod \langle c_i \rangle_i) : (\rightsquigarrow_C)(\prod \langle \Gamma_i \rangle_i)}$$

For the domains formed this way, we use $\langle\langle - \rangle\rangle = \prod \langle \dots \prod \langle - \rangle_{i \dots} \rangle_z$ for the functor $\mathbb{T}^{\Sigma \dots \Sigma n_{i \dots} \dots z} \rightarrow \mathbb{T}$, and may overload the term meta-context to refer to these functors.

A **generic rewrite** is a substitution of base rewrites into a rewrite constructor, written $d(\vec{l})$. A **specific rewrite** is a substitution of ground terms into a generic rewrite, $d(l(\vec{x}))$.

Rewrite constructors are the *reactive contexts* of Leifer and Milner: the substitution of base rewrites into a rewrite constructor is a morphism in the slice category $\mathbb{T}/Pr$. Their relative pushouts are the transitions that characterize behavior — once we have said which contexts are permitted to take part, which is the subject of §4.

Note 1. The combination of cartesian ($\times$) and closed ($\rightarrow$) structure on $\text{SubT} \rightarrow \mathbb{T}$ does not appear in the standard hierarchy of logical structures [Johnstone, 2002].

cartesian	regular	coherent	heyting	topos
$(\top, \wedge, =)$	$(\dots, \exists)$	$(\dots, \perp, \vee)$	$(\dots, \Rightarrow, \forall)$	$(\dots, \rightarrow, \Omega)$

Yet it is useful and natural. Arguably the simplest thing one could mean by *higher-order* reasoning is that in the same way we form types of functions, we can form *propositions about proofs*. Just as one curries $f : A \times B \rightarrow C$ to $A \rightarrow [B \rightarrow C]$, the same is useful for entailments. The basic rewrite of the π calculus is the entailment

$$\begin{array}{ccc} \top_{Nm}, \top_{Nm}, \top_{[Nm \setminus Pr]} & \vdash & (\rightsquigarrow) \\ \downarrow & & \downarrow \\ Nm, Nm, [Nm \setminus Pr] & \xrightarrow{o(a,n) | i(a, \lambda x.c), c[n]} & Pr, Pr \end{array}$$

which can be curried to a rewrite of *open* processes — continuations with a bound name.

$$\begin{array}{ccc} \top_{Nm}, \top_{[Nm \setminus Pr]} & \vdash & \top_{Nm} \Rightarrow (\rightsquigarrow) \\ \downarrow & & \downarrow \\ Nm, [Nm \setminus Pr] & \xrightarrow{\lambda a. (...)} & [Nm \setminus Pr, Pr] \end{array}$$

This is how we reason about rewrites under a lambda and within binding constructors. It is also why the transition system of §5 can be defined on open terms without the distinguished ground object of Leifer and Milner [2000].

Note 2. In a logic T , a subobject $U \multimap A$ is a *proposition* about a type; but U is also an object of T , hence a *type*. This is **comprehension**, or subset types.

$$\frac{x : A \vdash \varphi \text{ prop}}{\{x : A \mid \varphi\} \text{ type}}$$

It lets us define *partial operations* $p_{op} : \{\varphi\} \rightarrow C$, which we use in §3 to restrict replication to input-guarded processes. We do not use the corresponding refined-binding operations $r_{op} : [\{\varphi\} \rightarrow C] \rightarrow D$.

2.2 What is in the appendices, and a reading path

Two further pieces of the lambda-theory story are needed only in one place each, and both are now in appendices: the meta-theories $\mathbb{T}_\wedge$, $\mathbb{T}_\wedge^\rightarrow$ and their free extensions by left Kan extension, which §8 uses to say that a data-type encoding is *canonical*, are in Appendix B; the inference rules of the internal language are in Appendix A. A reader who wants the argument of this paper and not its foundations may go directly to §4, where the paper’s own definitions begin; §3 supplies the calculi those definitions are about.

3 The theories we treat

Our main theory is the **rho calculus**¹: the π -calculus with reflection between code and data, that is, between processes and names. Its basic rule is communication — an output process synchronizes with an input process along a name, transferring a list of processes which are converted into names and substituted into the input context — together with a rule for dereferencing a quoted process.

¹The name is an acronym: the *reflective higher-order* calculus. Dropping the periods from r.h.o. gives the English transliteration of the Greek letter following π , which is the pun we intend — the rho calculus comes after the π -calculus. We accordingly write *rho*, never ρ .

rho calculus

$$\begin{array}{c}
\vdash 0 : Pr \\
p_1 : Pr, p_2 : Pr \vdash p_1 | p_2 : Pr \\
p : Pr \vdash @ (p) : Nm \\
n : Nm \vdash *(n) : Pr \\
n : Nm, \vec{p} : Pr^k \vdash \text{out}_k(n, \vec{p}) : Pr \\
n : Nm, \lambda \vec{x}. q : [Nm^k \setminus Pr] \vdash \text{in}_k(n, \lambda \vec{x}. q) : Pr
\end{array}$$

$$\begin{array}{c}
p : Pr \vdash p | 0 = p \\
p_1 : Pr, p_2 : Pr \vdash p_1 | p_2 = p_2 | p_1 \\
p_1 : Pr, p_2 : Pr, p_3 : Pr \vdash (p_1 | p_2) | p_3 = p_1 | (p_2 | p_3) \\
n : Nm \vdash @(*(n)) = n
\end{array}$$

$$\begin{array}{c}
p_1 : Pr, p_2 : Pr \vdash (p_1 \rightsquigarrow p_2) \text{ prop} \\
n : Nm, \vec{p} : Pr^k, \lambda \vec{x}. q : [Nm^k \setminus Pr] | \top \vdash \text{out}_k(n, \vec{p}) | \text{in}_k(n, \lambda \vec{x}. q) \rightsquigarrow q[@\vec{p}/\vec{x}] \\
p : Pr | \top \vdash @(*(@ (p))) \rightsquigarrow p \\
p_1, p_2, q : Pr | (p_1 \rightsquigarrow p_2) \vdash p_1 | q \rightsquigarrow p_2 | q \\
p_1, p_2, q_1, q_2 : Pr | (p_1 \rightsquigarrow p_2), (q_1 \rightsquigarrow q_2) \vdash p_1 | q_1 \rightsquigarrow p_2 | q_2
\end{array}$$

These rewrites are drawn as string diagrams in Appendix D. Note that $*(@ (p)) \rightsquigarrow p$ is a rewrite whose redex is not of the form $t | u$; this is the one case the enumeration of Theorem 28 must handle separately.

Following Meredith and Radestock [2005], the calculus above has no name restriction: fresh names are obtained by quotation. Replication, however, we treat as an extension, because it is the smallest interesting instance of the pattern this paper is about — an extension whose defining rewrite is compiled away.

rho[!]: rho + replicated input

$$\begin{array}{c}
(p, n, c) : \{p : Pr, n : Nm, c : [Nm \setminus Pr] \mid p = \text{in}(n, c)\} \\
\vdash ! (p) : Pr, \\
! (p) \rightsquigarrow p | ! (p)
\end{array}$$

The comprehension of Note 2 is what lets us restrict $!$ to input-guarded processes.

π calculus (with replicated input)

$$\begin{array}{c}
\vdash 0 : Pr \\
n \in \text{Names} \vdash n : Nm \\
p_1 : Pr, p_2 : Pr \vdash p_1 | p_2 : Pr \\
n : Nm, k : Pr \vdash \text{out}(n, k) : Pr \\
n : Nm, \lambda x. q : [Nm \setminus Pr] \vdash \text{in}(n, \lambda x. q) : Pr \\
\lambda x. q : [Nm \setminus Pr] \vdash \nu(\lambda x. q) : Pr \\
p : Pr, n : Nm, c : [Nm \setminus Pr] \mid p = \text{in}(n, c) \vdash ! (p) : Pr
\end{array}$$

$$\begin{array}{lcl}
p : Pr & \vdash & p \mid 0 = p \\
p_1 : Pr, p_2 : Pr & \vdash & p_1 \mid p_2 = p_2 \mid p_1 \\
p_1 : Pr, p_2 : Pr, p_3 : Pr & \vdash & (p_1 \mid p_2) \mid p_3 = p_1 \mid (p_2 \mid p_3) \\
& \vdash & \nu(\lambda x.0) = 0 \\
p : Pr, \lambda x.q : [Nm \setminus Pr] & \vdash & p \mid \nu(\lambda x.q) = \nu(\lambda x.(p \mid q)) \\
\lambda xy.p : [Nm, Nm \setminus Pr] & \vdash & \nu(\lambda x.\nu(\lambda y.p)) = \nu(\lambda y.\nu(\lambda x.p)) \\
\\
p_1 : Pr, p_2 : Pr & \vdash & (p_1 \rightsquigarrow p_2) \text{ prop} \\
n : Nm, k : Nm, \lambda x.q : [Nm \setminus Pr] \mid \top & \vdash & \text{out}(n, k) \mid \text{in}(n, \lambda x.q) \rightsquigarrow q[k/x] \\
p_1, p_2, q : Pr \mid (p_1 \rightsquigarrow p_2) & \vdash & p_1 \mid q \rightsquigarrow p_2 \mid q \\
\lambda x.s, \lambda x.t : [Nm \setminus Pr] \mid (\lambda x.s (\top_{Nm} \Rightarrow \rightsquigarrow) \lambda x.t) & \vdash & \nu(\lambda x.s) \rightsquigarrow \nu(\lambda x.t) \\
p : Pr & \vdash & !(p) \rightsquigarrow p \mid !(p)
\end{array}$$

The three ν -equations are the ones that an encoding into rho will satisfy only up to bisimilarity, and they are the reason §5.9 is needed. The equations involve the strength of $[Nm \rightarrow -]$: for op any of the unary process operations,

$$\begin{array}{ccc}
\Gamma, [Nm \setminus Pr] & \xrightarrow{\Gamma, \nu} & \Gamma, Pr \\
\downarrow st & & \downarrow op \\
[Nm \setminus \Gamma, Pr] & & \\
\downarrow [Nm \setminus op] & & \downarrow \\
[Nm \setminus Pr] & \xrightarrow{\nu} & Pr
\end{array}$$

expressing $op(x, \nu(\lambda n.p)) = \nu(\lambda n.op(x, p))$; the commutation of ν with itself uses the symmetry $\sigma : A \times B \cong B \times A$.

Remark 4 (why the ambient calculus is not here). The ambient calculus [Bonchi et al., 2009] is the natural next example and we do not treat it. The reason is structural rather than incidental. Theorem 28 enumerates redex IPOs under the hypothesis that every unary rewrite constructor commutes with parallel composition. For ambients the constructor $n[-]$ does not:

$$n[p \mid q] \neq n[p] \mid q$$

Locality is exactly the failure of that commutation — it is what makes ambients ambients. Consequently the enumeration does not apply, and separately, Bonchi et al. [2009] show that the IPO-derived semantics for ambients is strictly finer than the intended one and has to be coarsened by barbs. Both facts point to a different construction, which we do not attempt here.

4 Admissible observers

Before deriving a transition system we must say which contexts are permitted to observe. This section shows that the answer is not an extra piece of syntax to be legislated, but is determined by a choice of equivalence at the ground types.

4.1 Observation relations

Definition 5. Let T be a lambda theory. An **observation relation** on T is a family $\approx_A$ of equivalence relations, one for each type A of T , subject to the clauses

$$\begin{aligned}
f \approx_{A \times B} g & \quad \text{iff} \quad \pi_1 f \approx_A \pi_1 g \text{ and } \pi_2 f \approx_B \pi_2 g \\
f \approx_{[A \setminus B]} g & \quad \text{iff} \quad a \approx_A a' \text{ implies } f(a) \approx_B g(a') \\
p \approx_{Pr} q & \quad \text{iff} \quad p \text{ and } q \text{ are bisimilar in the derived transition system}
\end{aligned}$$

and otherwise freely chosen at the remaining base types. We write $\approx$ for the whole family and drop the subscript when the type is clear.

The clauses for products and function types are those of a logical relation; the content of an observation relation is entirely in the choice made at the base types other than Pr .

Definition 6. An operation $op: \Gamma \rightarrow A$ of $\mathbf{T}$ is **admissible** for $\approx$ if it preserves the relation: $\gamma \approx_\Gamma \gamma'$ implies $op(\gamma) \approx_A op(\gamma')$. We write $\mathcal{D}(\approx)$ for the subcategory of $\mathbf{T}$ generated by the admissible operations, and call its morphisms the **observers**.

Proposition 7. $\mathcal{D}(\approx)$ contains all identities and projections and is closed under composition; and $\approx$ is a congruence for every context in $\mathcal{D}(\approx)$.

Proof. Identities and projections preserve $\approx$ by the product clause. If op and op' preserve $\approx$ then so does $op \circ op'$, since preservation is a closure condition on relations composed forwards. A context $c: \langle\langle Pr \rangle\rangle \rightarrow Pr$ in $\mathcal{D}(\approx)$ is by definition a composite of admissible operations, so preserves $\approx$; that is exactly the statement that $\approx$ is a congruence for c . $\square$

Proposition 7 looks tautological and in a sense it is: we have arranged for congruence to hold by defining the observers to be the operations for which it holds. The content is not in the proposition but in the fact that the definition of $\approx_{Pr}$ refers to the derived transition system, which in turn refers to $\mathcal{D}$. The two are mutually recursive, and we must check the recursion has a solution.

4.2 The circularity, and why it is respectable

The circle is worth stating in full before it is closed, because it is the load-bearing structure of the paper and it is easy to mistake for an equivocation. It runs: an observer class $\mathcal{D}$ determines a set of labels, hence a derived transition system, hence a bisimilarity $\approx$; and a bisimilarity $\approx$ determines which operations preserve it, hence an observer class $\mathcal{D}(\approx)$. Neither is prior. A construction that fixed one and derived the other would be making a choice it could not justify — and that is exactly what a construction which takes all contexts as observers is doing, silently.

The circle is not vicious because both halves run the same way.

Proposition 8. Fix a choice of $\approx_A$ at each base type other than Pr . Let Φ send a pair $(\mathcal{D}, \approx)$ to $(\mathcal{D}(\approx'), \approx')$, where $\approx'$ is bisimilarity for the $\mathcal{D}$ -relative transition system of Definition 18. Then Φ is monotone in the inclusion order on pairs, and so has a greatest fixed point.

Proof. Enlarging $\mathcal{D}$ admits more labels, hence more transitions to be matched, hence a *finer* bisimilarity: $\mathcal{D} \mapsto \approx$ is antitone. Coarsening $\approx$ makes preservation harder to satisfy, hence admits fewer operations: $\approx \mapsto \mathcal{D}(\approx)$ is antitone. The composite of two antitone maps is monotone, and the pairs form a complete lattice, so Knaster–Tarski applies. $\square$

We take the greatest fixed point throughout: the coarsest observation relation compatible with the largest class of observers preserving it. This is the sense in which labels are compared coinductively rather than syntactically — matching a label c against a label c' asks that $c \approx c'$, with $\approx$ the relation being defined.

Three remarks on what Proposition 8 buys.

First, it is what licenses the definition of $\mathcal{D}$ at all. Without it one would have to stipulate the observers, and every theorem in this paper would be relative to a stipulation. With it, the only free choice is the equivalence at the base types other than Pr ; everything else, including the exclusion of $@$ (Proposition 10), is computed.

Second, the greatest fixed point is the right one for the same reason that bisimilarity rather than some smaller relation is the right behavioural equivalence: we want to identify programs unless something distinguishes them, and to admit observers unless admitting one would distinguish programs that nothing else does. A least fixed point would give the finest relation compatible with the fewest observers, which is the empty answer.

Third, the two antitone maps are of quite different characters, and it is worth noticing that they compose. $\mathcal{D} \mapsto \approx$ is semantic: more observers, finer equivalence. $\approx \mapsto \mathcal{D}(\approx)$ is syntactic: it inspects the operations of the signature one at a time and asks a preservation question about each. The fixed point is therefore computable in the cases we care about — one starts from all operations, discovers which fail preservation, removes them, and recomputes — and §6 is that computation carried out for ρ , terminating after one round with $@$ removed.

4.3 The rho calculus: names are discrete

For the rho calculus we make the only reasonable choice at the base type Nm .

Definition 9. *The **standard observation relation** on rho takes $\approx_{Nm}$ to be equality.*

The justification is operational: the communication rewrite fires when two names are equal, and no coarser relation on names is respected by it. A calculus in which channels were matched up to bisimilarity of their quoted processes would have to search an equivalence class at every rendezvous, which is not an implementable rule.

This single choice determines the admissible operations, and in particular it rules one out.

Proposition 10. *For the standard observation relation on rho, the operation $@ : Pr \rightarrow Nm$ is not admissible.*

Proof. Admissibility of $@$ would require $p \approx_{Pr} q$ to imply $@(p) \approx_{Nm} @(q)$, that is $@(p) = @(q)$, that is $p = q$ by injectivity of quotation. So $@$ is admissible only if bisimilarity is equality. It is not: take

$$p = \text{out}(a, 0) \mid \text{in}(b, \lambda x. 0) \quad q = \text{in}(b, \lambda x. 0) \mid \text{out}(a, 0)$$

which are distinct terms only if we do not quotient by the commutativity equation; for a genuine separation take instead any $p \rightsquigarrow^* q$ with $p \neq q$, for instance $p = *(@ (0))$ and $q = 0$, which are weakly bisimilar and not equal. Then $\text{out}(@ (p), z)$ and $\text{out}(@ (q), z)$ are not bisimilar, since they output on distinct channels and no context can match one action with the other. $\square$

Corollary 11. *For rho with the standard observation relation, the admissible operations include $\text{out}(n, -)$, $\text{in}(n, -)$, $- \mid -$ and $*(@ (-))$, and exclude every operation with a hole under $@$ alone, in particular $\text{out}(@ (-), z)$ and $\text{in}(@ (-), z)$.*

Note that $*(@ (-))$ is admissible although $@$ is not: the composite lands back in Pr , where the relation is bisimilarity, and $*(@ (p)) \rightsquigarrow p$. Admissibility is a property of composites, not of each factor — which is precisely why $\mathcal{D}(\approx)$ must be described as the subcategory *generated by* admissible operations, and why decomposition-closure (§5) is a real hypothesis rather than a formality.

4.4 Encodings: the observation relation transported

The second source of observation relations is an encoding. If $\llbracket - \rrbracket : S \rightarrow T$ maps types of S to types of T , then the observation relation of S can be transported forwards.

Definition 12. *Let $\llbracket - \rrbracket : S \rightarrow T$ send Pr_S to $\langle Pr_T \rangle$ for a meta-context $\langle - \rangle$. The **transported observation relation** $\llbracket \approx \rrbracket$ on the image is generated by*

$$\llbracket p \rrbracket \llbracket \approx_A \rrbracket \llbracket q \rrbracket \quad \text{iff} \quad p \approx_A^S q$$

and its admissible operations are the images $\llbracket op \rrbracket$ of the operations of S . We write $\llbracket \mathcal{D} \rrbracket$ for the subcategory they generate — the image of the source contexts.

This is the formal content of the slogan that an encoding may only be probed by encoded contexts. It is not a restriction imposed from outside: it is what Definition 5 yields when the relation at the base types is the one the source theory already had. The remainder of the paper is the study of $\mathcal{D}$ -relative behavior for these two families of examples.

5 Relative pushouts, relatively

Beginning with Milner [1982], process behavior has been specified by a labelled transition system: a relation $\rightarrow \subset T \times A \times T$ for which $t \xrightarrow{a} u$ means that when t makes the action a it becomes u . This converts internal term structure into external graph structure, revealing how a program can act in different environments.

$$\text{out}(n, p) \xrightarrow{- \mid \text{in}(n, \lambda x. c)} c(@p)$$

Such systems had to be made by hand, relying on the researcher to know which labels give the correct view of behavior. Leifer and Milner [2000] showed how to derive them: labels are the minimal contexts enabling a rewrite, minimality being a universal property.

5.1 Relative pushouts

Definition 13. Let $\mathbf{C}$ be a category and T an object. In the **slice category** $\mathbf{C}/T$ an object is a morphism $t : X \rightarrow T$ and a morphism $f : t \rightarrow u$ is a commuting triangle $t = u \circ f$.

$$\begin{array}{ccc} X & \xrightarrow{f} & Y \\ & \searrow t & \swarrow u \\ & T & \end{array}$$

A **relative pushout (RPO)** in $\mathbf{C}$ is a pushout in some $\mathbf{C}/T$: given $x : z \rightarrow t$ and $y : z \rightarrow u$, the pair p with $i : t \rightarrow p$ and $j : u \rightarrow p$ is an RPO if for every other candidate (q, i', j') there is a unique $h : p \rightarrow q$ with $i' = hi$ and $j' = hj$.

$$\begin{array}{ccccc} \Gamma & \xrightarrow{x} & A & \equiv & A \\ & \searrow z & \swarrow t & & \downarrow i \\ & & T & & \\ & \swarrow u & \nwarrow p & & \downarrow q \\ B & \xrightarrow{j} & C & & \\ \parallel & & & & \downarrow i' \\ B & \xrightarrow{j'} & D & & \end{array}$$

$\exists! h$

An RPO is **idempotent** (an IPO) if p is an identity.

Because morphisms of a slice category are factorizations, an RPO is the *greatest common factor* of an equal pair of substitutions $t[f] = u[g]$; a transition is asserted when the redex and the context are *relatively prime*.

5.2 Two things Leifer and Milner did not need

The construction of Leifer and Milner [2000] is stated for *reactive systems*, which carry two restrictions we must examine.

First, a reactive system has a distinguished object 0 that is the domain of all IPOs, which in effect restricts the theory to ground terms. Their proofs do not use this object. We therefore adopt

an *open* form of the construction, on arbitrary contexts Γ , and Note 1 of §2 is what makes this coherent: currying an entailment produces a rewrite of open processes, so there is no need to close a term before observing it.

Second, a reactive system carries a subcategory $\mathcal{D}$ of *reactive* contexts — those in which a rewrite may occur — closed under composition and decomposition. Their proofs do use this, and they use nothing about the ambient category beyond it. This is the observation we build on: their arguments take place entirely inside $\mathcal{D}$, so they may be run in $\mathcal{D}$. What we change is which restriction $\mathcal{D}$ expresses. Leifer and Milner restrict where reaction may happen; we additionally restrict who may look.

Definition 14. *Let T be a lambda theory and $\mathcal{D} \subseteq T$ a subcategory of observers. A **$\mathcal{D}$ -relative pushout** for $x : z \rightarrow t$, $y : z \rightarrow u$ is an RPO computed in the slice $\mathcal{D}/Pr$: all four legs and the mediating morphism h are required to lie in $\mathcal{D}$. A **$\mathcal{D}$ -relative IPO** is one whose $\mathcal{D}$ -RPO is an identity.*

The point of the relativization is that $\mathcal{D}$ -relative IPOs and IPOs are genuinely different, and it is the relative notion that is wanted.

Example 15. Let $\llbracket - \rrbracket : \pi \rightarrow \text{rho}$ be the encoding of §7, and consider a transition out of $\llbracket p \rrbracket$. The rho-minimal context enabling a communication is a bare $- | \text{out}(a, q)$. No such context is of the form $\llbracket c \rrbracket$: every encoded context carries the name-supply parameters (n, v) and the replication server $!N$, which rho-minimality strips away. Insisting on minimality in rho therefore forces observation from outside the image; insisting on encoded contexts abandons minimality. Relative minimality asks for the smallest *encoded* context, and keeps both.

Definition 16. *Let T be a lambda theory and $\mathcal{D}$ a subcategory of observers. T **has all $\mathcal{D}$ -redex-RPOs** if for every redex dl , seen as a morphism $l : dl \rightarrow d$ in $\mathcal{D}/Pr$, the $\mathcal{D}$ -relative pushout along l exists for all $t : dl \rightarrow c$ with $c \in \mathcal{D}$.*

5.3 Which contexts are observations

We can now finish a definition that earlier drafts of this material left open. The question “which contexts are observations?” has no absolute answer; it has an answer relative to $\approx$.

Definition 17. *Let $\approx$ be an observation relation on T . A context $c : \langle \vec{Pr} \rangle_1 \rightarrow \langle \vec{Pr} \rangle_2$ is an **observation** if $c \in \mathcal{D}(\approx)$ and $\mathcal{D}(\approx)$ is closed under decomposition up to $\approx$: whenever $c = c_1 \circ c_2$ in T , there are $d_1, d_2 \in \mathcal{D}(\approx)$ with $d_1 \approx c_1$, $d_2 \approx c_2$ and $d_1 \circ d_2 \approx c$.*

Decomposition-closure is what Leifer and Milner [2000] require of their reactive contexts, and it is needed for the same reason: the congruence argument factors a context and must know that the factors are still legitimate. Requiring it only up to $\approx$ rather than on the nose is forced on us by the examples — see Obligation 26.

5.4 The derived transition system

Recall that rewrite operations have the form $op : \prod \langle Pr \rangle_i \rightarrow Pr$, so their composites have the form $d : \langle \vec{Pr} \rangle \rightarrow Pr$. We assume these closed under decomposition, so that every factorization of d is $f \langle \vec{e} \rangle$ with f a rewrite constructor and $\vec{e}$ a tree of rewrite constructors; hence any first factor is of the form $\langle \vec{d} \rangle$.

Definition 18. *Let T be a lambda theory with all $\mathcal{D}$ -redex-RPOs. The **$\mathcal{D}$ -relative derived transition system** $T[\rightarrow_{\mathcal{D}}]$ is defined as follows. For each meta-context $\langle - \rangle$, each tree of rewrite constructors $\vec{d}$ of type $\langle \vec{Pr} \rangle$, and each substitution of base rewrites $[[\Gamma_i \vdash l_i \rightsquigarrow r_i]_j \cdots]_z$, then for*

each $\mathcal{D}$ -relative IPO of the form below with c an observation, assert $\Gamma \vdash \vec{t} \xrightarrow{\mathcal{D}} \langle\langle d \langle \vec{\tau} \rangle \rangle\rangle$.

$$\begin{array}{ccc} \langle\langle \langle \vec{\tau} \rangle \rangle \rangle & \xrightarrow{\vec{t}} & \langle\langle \vec{Pr} \rangle\rangle_0 \\ \downarrow & & \downarrow c \\ \langle\langle \langle \vec{\tau} \rangle \rangle \rangle & & \langle\langle \vec{Pr} \rangle\rangle \\ \downarrow & & \downarrow \\ \langle\langle \langle \vec{P} \rangle \rangle \rangle & \xrightarrow{\langle\langle \vec{d} \rangle \rangle} & \langle\langle \vec{Pr} \rangle\rangle \end{array}$$

Additionally, for each substitution $x : 1 \rightarrow \langle\langle \langle \vec{\tau} \rangle \rangle \rangle$ into the above diagram, assert $\cdot \vdash \vec{t}x \xrightarrow{\mathcal{D}} \langle\langle d \langle \vec{\tau} \rangle \rangle \rangle x$.

Definition 19. A $\mathcal{D}$ -bisimulation is a relation $\sim_{\mathcal{D}} \subset Pr \times Pr$ such that $p_0 \sim_{\mathcal{D}} q_0$ implies that every $p_0 \xrightarrow{\mathcal{D}} p_1$ is matched by some $q_0 \xrightarrow{\mathcal{D}} q_1$ with $c \approx c'$ and $p_1 \sim_{\mathcal{D}} q_1$, and symmetrically. The **weakening** $\rightarrow_{\mathcal{D}*}$ is

$$p \xrightarrow{\mathcal{D}*} q := \exists \hat{p}, \hat{q}. (p \rightsquigarrow^* \hat{p}) \wedge (\hat{p} \xrightarrow{\mathcal{D}} \hat{q}) \wedge (\hat{q} \rightsquigarrow^* q)$$

and **weak $\mathcal{D}$ -bisimilarity** $\approx_{\mathcal{D}}$ is defined in the free infinitary theory of Proposition 80 by

$$\begin{aligned} (p \approx_{\mathcal{D}}^0 q) &:= \top \\ (p \approx_{\mathcal{D}}^{n+1} q) &:= \forall p'. \forall c. (p \xrightarrow{\mathcal{D}} p') \Rightarrow \exists q', c'. (c \approx c') \wedge (q \xrightarrow{\mathcal{D}*} q') \wedge (p' \approx_{\mathcal{D}}^n q') \wedge \\ &\quad \forall q'. \forall c. (q \xrightarrow{\mathcal{D}} q') \Rightarrow \exists p', c'. (c \approx c') \wedge (p \xrightarrow{\mathcal{D}*} p') \wedge (p' \approx_{\mathcal{D}}^n q') \\ (p \approx_{\mathcal{D}} q) &:= \bigwedge_{i \in \mathbb{N}} p \approx_{\mathcal{D}}^i q \end{aligned}$$

At types other than Pr , $\approx_{\mathcal{D}}$ is the observation relation of Definition 5; this is what gives meaning to $\Gamma \vdash t \approx_{\mathcal{D}} u$ for open terms and to $\lambda x. c \approx_{\mathcal{D}} \lambda x. d$ at $[Nm \setminus Pr]$.

Matching labels up to $\approx$ rather than by syntactic identity is not a refinement we could avoid. Two encoded contexts that differ only in the internal name of a fresh channel must be allowed to match, and they are equal only up to bisimilarity.

5.5 Normality and substitution

Definition 20. T is **rw-normal** if every ground rewrite is of the form $e \circ x : 1 \rightarrow \Gamma \rightarrow Rw$ for a generic rewrite e , and **eq-normal** if every ground equation is $lx = rx$ for an equation (l, r) derived from the presentation. T is **normal** if both hold.

Normality is what lets us pass between generic and specific rewrites, and it is doing a job that in the wider literature is done by moving to a 2-categorical setting; see Remark 30. Henceforth we assume all theories normal.

Proposition 21. Suppose $\Gamma \vdash \vec{t} \approx_{\mathcal{D}} \vec{u}$. Then for any $x : 1 \rightarrow \Gamma$, $\cdot \vdash \vec{t}x \approx_{\mathcal{D}} \vec{u}x$.

Proof. Let $\cdot \vdash \vec{t}x \xrightarrow{\mathcal{D}} z$. By eq-normality $\vec{t}x = \langle\langle d \langle \vec{\tau} \rangle \rangle \rangle x$, and by rw-normality $z = \langle\langle d \langle \vec{\tau} \rangle \rangle \rangle x$, so the transition is determined by the diagram

$$\begin{array}{ccccc} 1 & \xrightarrow{x} & \langle\langle \langle \vec{\tau} \rangle \rangle \rangle & \xrightarrow{\vec{t}} & \langle\langle \vec{Pr} \rangle\rangle_0 \\ & & \downarrow & & \downarrow c \\ & & \langle\langle \langle \vec{\tau} \rangle \rangle \rangle & & \langle\langle \vec{Pr} \rangle\rangle \\ & & \downarrow & & \downarrow \\ & & \langle\langle \langle \vec{P} \rangle \rangle \rangle & \xrightarrow{\langle\langle \vec{d} \rangle \rangle} & \langle\langle \vec{Pr} \rangle\rangle \end{array}$$

Because $\vec{t} \approx_{\mathcal{D}} \vec{u}$ there are $\vec{u} \rightsquigarrow^* u' \xrightarrow{\vec{c}'}_{\mathcal{D}} v' \rightsquigarrow^* z'$ with $c \approx c'$ and $z' \approx_{\mathcal{D}} z$, so there is a $\mathcal{D}$ -IPO for u' matching the one above; substituting into $(\rightsquigarrow)$ gives $\vec{u}x \rightsquigarrow^* u'x \xrightarrow{\vec{c}'}_{\mathcal{D}} v'x \rightsquigarrow^* z'x$. Hence $\{(fx, gx) \mid f \approx_{\mathcal{D}} g\}$ is a $\mathcal{D}$ -bisimulation. $\square$

5.6 Congruence

Lemma 22 (Leifer and Milner, 2000, Proposition 2, relativized). *Suppose $\mathcal{D}$ is closed under composition and decomposition up to $\approx$. If the outer rectangle of a pasting is a $\mathcal{D}$ -IPO and the left square is a $\mathcal{D}$ -RPO, then the right square is a $\mathcal{D}$ -IPO.*

Proof. The statement and proof of Leifer and Milner [2000] concern only the slice category in which the pushouts are computed, and use no property of the ambient category. Taking that slice to be $\mathcal{D}/Pr$, which is a category by closure under composition, the argument goes through verbatim; decomposition-closure up to $\approx$ is what guarantees that the factors produced in the course of the argument again lie in $\mathcal{D}$, up to the relation along which labels are matched. $\square$

Theorem 23 ($\mathcal{D}$ -relative congruence). *Let T have all $\mathcal{D}$ -redex-RPOs, with $\mathcal{D}$ closed under composition and decomposition up to $\approx$. Then $\mathcal{D}$ -relative strong bisimilarity is a congruence for $\mathcal{D}$: if $\Gamma \vdash t \sim_{\mathcal{D}} u$ then $\Gamma \vdash kt \sim_{\mathcal{D}} ku$ for every $k \in \mathcal{D}$.*

Proof. We show $S = \{(kt, ku) \mid t \sim_{\mathcal{D}} u, k \in \mathcal{D}\}$ is a $\mathcal{D}$ -bisimulation. Let $kt \xrightarrow{\vec{c}}_{\mathcal{D}} d\langle\vec{r}\rangle$ with $c \in \mathcal{D}$; then the rectangle $c \circ (k, \Delta) = d \circ \langle\vec{l}\rangle$ is a $\mathcal{D}$ -IPO. Since T has all $\mathcal{D}$ -redex-RPOs, the $\mathcal{D}$ -RPO of (t, Δ) against $\langle\vec{l}\rangle$ exists; it is the left square, with legs c' and d' in $\mathcal{D}$, and by Lemma 22 the right square is a $\mathcal{D}$ -IPO. The left square being a $\mathcal{D}$ -IPO gives a transition $t \xrightarrow{\vec{c}'}_{\mathcal{D}} d'\langle\vec{r}\rangle$.

Because $t \sim_{\mathcal{D}} u$ there is $u \xrightarrow{\vec{c}'}_{\mathcal{D}} e'\langle\vec{r}'\rangle$ with $c' \approx c''$ and $e'\langle\vec{r}'\rangle \sim_{\mathcal{D}} d'\langle\vec{r}\rangle$. This determines a second left $\mathcal{D}$ -IPO with the same right-hand leg up to $\approx$; pasting it with the right square, which is a $\mathcal{D}$ -IPO and lies in $\mathcal{D}$, yields $ku \xrightarrow{\vec{c}}_{\mathcal{D}} ie'\langle\vec{r}'\rangle$ where i is the mediating morphism. Since $kt \xrightarrow{\vec{c}}_{\mathcal{D}} d\langle\vec{r}\rangle = id'\langle\vec{r}\rangle$ and $d'\langle\vec{r}\rangle \sim_{\mathcal{D}} e'\langle\vec{r}'\rangle$, the pair (kt, ku) has matching transitions with successors again in S . $\square$

The shape of this statement is the whole point. It does not say that bisimilarity is a congruence; it says that $\mathcal{D}$ -relative bisimilarity is a congruence for $\mathcal{D}$. In §6 this is the difference between a theorem with an embarrassing caveat about reflection and a theorem whose hypothesis names the caveat; in §7 it is the difference between an encoding that fails to preserve behavior and one that preserves it against the observers that matter.

Theorem 24 (weak congruence). *OBLIGATION Under the hypotheses of Theorem 23, together with the condition that every context of $\mathcal{D}$ is either reactive or IPO-uniform in the sense of Di Gianantonio et al. [2009, Section 3], weak $\mathcal{D}$ -bisimilarity is a congruence for $\mathcal{D}$.*

We flag this as an obligation rather than a theorem. Weak bisimilarity is not in general a congruence for all contexts, and it is not yet clear to us whether the failure is an artifact of the framework or a fact of life about weak equivalences; Di Gianantonio et al. [2009] give sufficient conditions in the second-order setting and we expect them to transfer, but we have not verified the transfer. This matters more than it may appear: the results of §7 and §8 are all statements about *weak* bisimilarity, since an encoding replaces a one-step operation by a multi-step protocol. For rho specifically we give a direct proof in Theorem 34 that does not depend on the general result.

5.7 Existence of relative IPOs

Proposition 25. *SKETCH Let T have all redex-RPOs and let $\mathcal{D}$ be closed under composition and decomposition up to $\approx$. Suppose further that $\mathcal{D}$ **reflects factorizations**: whenever $c \in \mathcal{D}$ and $c = f \circ g$ in T , there are $f', g' \in \mathcal{D}$ with $f \approx f'$ and $g \approx g'$. Then T has all $\mathcal{D}$ -redex-RPOs, and the $\mathcal{D}$ -IPO of a square is the $\approx$ -least $\mathcal{D}$ -candidate.*

Proof sketch. Given a redex $\langle\langle l \rangle\rangle$ and t , the $\mathcal{D}$ -candidates (c, d) with $c \in \mathcal{D}$ form a preorder under factorization. Reflection of factorizations makes this preorder closed under the greatest common factor computed in T : if (c_1, d_1) and (c_2, d_2) are $\mathcal{D}$ -candidates, their T-RPO has legs that factor c_1 and c_2 , and by reflection those factors are $\approx$ -represented in $\mathcal{D}$. Hence the preorder is codirected and, T having all redex-RPOs, has a least element up to $\approx$; that element is the $\mathcal{D}$ -IPO. The gap in this argument is that codirectedness gives a lower bound for each pair but not for an infinite family; a well-foundedness condition on $\mathcal{D}$ is needed and we have not identified the right one. $\square$

Reflection of factorizations is a sharpening of what is elsewhere called *hosting* or faithfulness of an encoding: it asks not merely that source terms embed faithfully, but that the way a source context comes apart is visible in the target. It is exactly the hypothesis the encoding results of §7 consume.

Obligation 26. Verify that $\llbracket C_\pi \rrbracket \subseteq \text{rho}$ is decomposition-closed up to $\approx$.

This is the first thing to check and we expect it to fail on the nose. A rho-factorization of an encoded context can cut a constructed name such as $@(\text{o}(n, 0) \mid \text{i}(n, \lambda x. 0))$ into pieces that are not encodings of anything. The question is whether every such factorization is $\approx$ -equivalent to one through $\llbracket C_\pi \rrbracket$. If the answer is no, the closure of $\llbracket C_\pi \rrbracket$ must be computed and shown still to exclude the discriminating contexts of Proposition 10; if the closure swallows them, the approach fails and we would want to know that early.

There is now a route. For the encoding of Theorem 43 the translation of a context is literally a context of the target and $\llbracket c(p) \rrbracket = \llbracket c \rrbracket(\llbracket p \rrbracket)$ holds *on the nose*, because an encoded term is an abstraction over an address and plugging is instantiation [Meredith, 2026a]. Decomposition-closure for that encoding therefore reduces to a statement about which sub-abstractions of $\llbracket c \rrbracket$ are themselves encodings of π contexts, which is a syntactic question about the six clauses of the translation rather than a question about arbitrary rho terms. We expect this to be the cheapest of the outstanding obligations to settle, and we recommend settling it first.

5.8 Enumerating redex IPOs

Definition 27. For each rewrite $\Gamma \vdash l \rightsquigarrow r$, l is a **process redex** if there is a nontrivial factorization $l = t(p, \Delta)$ for $p : Pr$ and $\Delta \subset \Gamma$. Note that $*(@ (p)) \rightsquigarrow p$ has no such factorization.

Theorem 28. SKETCH Let $(T, Pr, \rightsquigarrow)$ be a lambda theory in which $(Pr, - \mid -, 0)$ is a commutative monoid, every basic process redex has the form $t \mid u$, the only non-unary rewrite constructor is $- \mid -$, and every unary rewrite constructor op commutes with parallel composition:

$$\begin{array}{ccc}
 Pr, \langle Pr \rangle & \xrightarrow{Pr, op} & Pr, Pr \\
 st \downarrow & & \downarrow - \mid - \\
 \langle Pr, Pr \rangle & & \\
 \langle - \mid - \rangle \downarrow & & \downarrow \\
 \langle Pr \rangle & \xrightarrow{op} & Pr
 \end{array}$$

Then T has all redex IPOs, given by the basic factorizations together with one inference rule per unary constructor. For each $l \rightsquigarrow r$ and $l = ct$ with $t \neq \text{id}_{Pr}$ we have $\Gamma \vdash t \xrightarrow{c} r$:

$$\begin{array}{ccc}
 \Gamma & \xrightarrow{t, \Delta} & Pr, \Delta \\
 l \downarrow & & \downarrow c \\
 Pr & \xlongequal{\quad} & Pr
 \end{array}$$

and for each op with $l \rightsquigarrow r \vdash op\langle l \rangle \rightsquigarrow op\langle r \rangle$, since $l = t \mid u$ and the square above is an IPO, pasting gives

$$\begin{array}{ccccc}
& & \frac{\Gamma \vdash x \xrightarrow{t' \mid u'} r}{\langle \Gamma \rangle \vdash op(x) \xrightarrow{t' \mid u'} op(r)} & & \\
\langle \Gamma \rangle & \xrightarrow{\langle x, \Delta \rangle} & \langle Pr, \Delta \rangle & \xrightarrow{op, Pr} & Pr, Pr \\
\downarrow \langle l \rangle & & \downarrow \langle t' \mid u' \rangle & & \downarrow t' \mid u' \\
\langle Pr \rangle & \xlongequal{\quad} & \langle Pr \rangle & \xrightarrow{op} & Pr
\end{array}$$

Corollary 29. π satisfies the hypotheses of Theorem 28: its only unary rewrite constructor is ν , and the equation $p \mid \nu(\lambda x.q) = \nu(\lambda x.(p \mid q))$ is exactly the required commutation.

For rho the hypotheses hold for the communication rewrite but not for dereferencing, whose redex $\ast(@p)$ is not a parallel composition. That rewrite contributes the single additional transition $\ast(@p) \xrightarrow{\quad} p$, with the identity context as label; the label is an observation because $\ast(@(-))$ is admissible by Corollary 11, even though $@$ alone is not. Completing the case analysis for rho is what makes this theorem a sketch rather than a proof.

Remark 30 (strict pushouts and the 2-categorical alternative). It is well known that strict RPOs frequently fail to exist in the presence of structural congruence, and the standard repair is to move to a bicategorical setting and use *groupoidal* relative pushouts [Sassone and Sobociński, 2003, 2005]. The commutative-monoid equations on $- \mid -$ put us squarely in that territory. Our normality assumption is doing the same work by other means, and we regard the relationship as unresolved: either normality should be justified as a hypothesis a MeTTaIL presentation can be checked against, or the construction should be redone with GRPOs and normality dropped. We note that our matching of labels up to $\approx$ rather than on the nose is already a step in the 2-categorical direction, since it replaces equality of labels by a coherent isomorphism.

5.9 Working up to bisimilarity

There is a second relativization we need, and it turns out to be the same one.

An encoding will in general fail to preserve the equations of its source on the nose. The ν -equations of π are the standard example: $\llbracket p \mid \nu(\lambda x.q) \rrbracket$ and $\llbracket \nu(\lambda x.(p \mid q)) \rrbracket$ are distinct rho terms that are bisimilar. An earlier approach to this problem replaced the equations of the source by an internal congruence Eq and asked for relative pushouts with respect to it. That construction requires rebuilding the framework and we do not pursue it, because the following suffices.

Definition 31. Let $\mathcal{D}$ be a subcategory of observers and $\approx$ the observation relation. The category $\mathcal{D}/\approx$ has the objects of $\mathcal{D}$ and $\approx$ -classes of morphisms as morphisms. Composition is well defined because $\approx$ is a congruence for $\mathcal{D}$ (Proposition 7).

Proposition 32. A functor $S \rightarrow \mathcal{D}/\approx$ is the same thing as a map $\llbracket - \rrbracket$ of presentations such that $p =_S q$ implies $\llbracket p \rrbracket \approx \llbracket q \rrbracket$. Relative pushouts computed in $\mathcal{D}/\approx$ are the $\mathcal{D}$ -relative pushouts of Definition 14 with labels matched up to $\approx$.

Proof. The first claim is the universal property of the quotient: a functor out of S must send equal terms to equal morphisms of $\mathcal{D}/\approx$, and equality there is $\approx$. The second is Definition 31 unwound, since a candidate in the quotient is an $\approx$ -class of candidates in $\mathcal{D}$. $\square$

So the single move of working in $\mathcal{D}/\approx$ handles both restrictions at once: the subcategory says which contexts may observe, and the quotient says that they observe only up to bisimilarity. Every encoding result below is stated in $\mathcal{D}/\approx$, and the clause “equations become bisimulations” is not an extra demand on encodings but the definition of a functor into the quotient.

6 First instance: reflection in the rho calculus

The rho calculus is the case in which the observer subcategory is proper for a reason internal to the theory. We record the structural property that makes its behavior tractable, and then prove congruence for the admissible contexts of Corollary 11.

Definition 33. A lambda theory is **transparent** if for every context c that is not reactive there is a unique context $\bar{c}$ such that

$$c(t) \xrightarrow{\bar{c}} d(t)$$

where d is constructed from the variables of $\bar{c}$ and does not involve the application of c to t .

Transparency says that a non-reactive context can always be peeled off by exactly one complementary observation. In rho the pairing is visible immediately:

$$\begin{aligned} \text{out}(n, p) &\xrightarrow{-|\text{in}(n, c)} c(@p) \\ \text{in}(n, c) &\xrightarrow{-|\text{out}(n, p)} c(@p) \\ *(@p) &\xrightarrow{-} p \end{aligned}$$

so that $\overline{\text{out}(n, -)} = -|\text{in}(n, c)$ and dually, and $*(@(-))$ is its own complement with the identity label. Transparency is what makes the direct argument below possible without appeal to Theorem 23, and hence without depending on the outstanding Obligation 26.

Theorem 34. In the rho calculus with the standard observation relation, weak $\mathcal{D}$ -bisimilarity is a congruence for the operations generated by

$$\begin{aligned} \text{out}(n, -) &\quad Pr \rightarrow Pr \\ \text{in}(n, -) &\quad [Nm \setminus Pr] \rightarrow Pr \\ -|- &\quad Pr, Pr \rightarrow Pr \\ *(@(-)) &\quad Pr \rightarrow Pr \end{aligned}$$

when observations are restricted to the same class.

Proof. Let $p \approx_{\mathcal{D}} q$ and consider a transition out of $\text{out}(n, p)$. Because p cannot rewrite within out , the only transition is

$$\text{out}(n, p) \xrightarrow{-|\text{in}(n, c)}_{\mathcal{D}} c(@p)$$

matched by $\text{out}(n, q) \xrightarrow{-|\text{in}(n, c)}_{\mathcal{D}} c(@q)$; the result follows by structural induction, the successors being related because $\approx_{\mathcal{D}}$ at $[Nm \setminus Pr]$ is the logical relation of Definition 5.

Let $\lambda x.c \approx_{\mathcal{D}} \lambda x.d : [Nm \setminus Pr]$ and consider a transition out of $\text{in}(n, \lambda x.c)$. Because $\lambda x.c$ cannot rewrite within in , the only transition is

$$\text{in}(n, \lambda x.c) \xrightarrow{-|\text{out}(n, v)}_{\mathcal{D}} c(@v)$$

matched by $\text{in}(n, \lambda x.d) \xrightarrow{-|\text{out}(n, v)}_{\mathcal{D}} d(@v)$, and $c(@v) \approx_{\mathcal{D}} d(@v)$ by the function-type clause together with Proposition 21.

Let $p_1 \approx_{\mathcal{D}} q_1$ and $p_2 \approx_{\mathcal{D}} q_2$ and consider $p_1 | p_2 \xrightarrow{f}_{\mathcal{D}} t$. If $f = (-)$ then by transparency $p_1 = \text{op}(n, -) | \hat{p}_1$ and $p_2 = \overline{\text{op}}(n, -) | \hat{p}_2$, so

$$p_1 \xrightarrow{-|\overline{\text{op}}(n, -)}_{\mathcal{D}} \hat{p}_1 \quad \text{and} \quad p_2 \xrightarrow{-|\text{op}(n, -)}_{\mathcal{D}} \hat{p}_2$$

matched by weak transitions

$$q_1 \rightsquigarrow^* u \xrightarrow{\text{op}(n, -)}_{\mathcal{D}} \hat{u} \rightsquigarrow^* \hat{q}_1 \quad \text{and} \quad q_2 \rightsquigarrow^* v \xrightarrow{\overline{\text{op}}(n, -)}_{\mathcal{D}} \hat{v} \rightsquigarrow^* \hat{q}_2$$

and thus $q_1 \mid q_2 \rightsquigarrow^* u \mid v \xrightarrow{\mathcal{D}} \hat{u} \mid \hat{v} \rightsquigarrow^* \hat{q}_1 \mid \hat{q}_2$. If $f \neq (-)$ then it is $- \mid \text{out}(n, v)$ or $- \mid \text{in}(n, c)$ for communication with p_1 or p_2 , matched by $\rightsquigarrow^* \xrightarrow{\text{op}}_{\mathcal{D}} \rightsquigarrow^*$ on the corresponding side. So $q_1 \mid q_2$ matches every $\rightsquigarrow$ with $\rightsquigarrow^*$ and every $\xrightarrow{\mathcal{D}}$ with $\rightsquigarrow^* \xrightarrow{\mathcal{D}} \rightsquigarrow^*$, giving $p_1 \mid p_2 \approx_{\mathcal{D}} q_1 \mid q_2$.

Finally $*(@(-))$ preserves $\approx_{\mathcal{D}}$ because $*(@(p)) \rightsquigarrow p$ and $*(@(q)) \rightsquigarrow q$, so any transition of one is matched by the other after a silent step. $\square$

Remark 35. The theorem is stated for the admissible class and is false outside it, by Proposition 10. Earlier drafts recorded this as a caveat — “there is just one problem, reflection” — and then restricted the theorem by hand. In the present framework the restriction is not an afterthought: $\mathcal{D}$ was defined so that $@$ falls outside it, and the theorem is an instance of the general shape of Theorem 23. The rho calculus is not defective; it is a theory whose observer subcategory is proper.

Remark 36 (the inadmissibility of $@$, quantitatively). Proposition 10 says that $@$ manufactures observations. It is worth recording how fast. Meredith [2026a] proves three facts about the calculus itself, with no encoding in sight: a reduction adds at most one name to the set of names occurring in name position, so n pairwise distinct new names cost at least n rendezvous; a reduction raises the maximum quote depth in play by at most one, so a name k levels above what a process holds costs at least k ; and substitution reaches the object of a lift but never the interior of a quote. Names in rho are therefore a metered resource, and $@$ is the meter. This sharpens two things stated elsewhere in this paper. It is the quantitative content of the separation result of §7.6 — π has no meter at all, since its names have no structure to build — and it is why the name discipline of §8.3 must be checked rather than assumed: an encoding that expects to write a name it has not paid a rendezvous for is making an error the calculus will not forgive.

7 Second instance: encodings

7.1 What an encoding must preserve

The naive demand on a translation is that it be a functor of lambda theories preserving bisimilarity on the nose. Written out, this asks that $\llbracket - \rrbracket$ preserve products, pullbacks and function types, map Pr_S into $\langle Pr_T \rangle$ for a meta-context $\langle - \rangle$, map rewriting to a silent path, and satisfy $p \approx q \vdash \llbracket p \rrbracket \approx \llbracket q \rrbracket$ where the right-hand $\approx$ is bisimilarity in T .

This demand is not merely hard to meet; it is the wrong demand, and Example 15 says why. The target has observers the source never had. We therefore ask instead for a functor into $\mathcal{D}/\approx$, and Proposition 32 tells us what that amounts to.

Definition 37. Let S, T be lambda theories. An **encoding** $\llbracket - \rrbracket : S \rightarrow T$ is a functor $S \rightarrow \llbracket \mathcal{D} \rrbracket / \approx$, where $\llbracket \mathcal{D} \rrbracket$ is the transported observer subcategory of Definition 12. Explicitly, $\llbracket - \rrbracket$

- preserves conjunction and substitution, that is products and pullbacks;
- preserves function types;
- maps Pr_S to $\langle Pr_T \rangle$ for some meta-context $\langle - \rangle$;
- sends each equation of S to a bisimulation: $p =_S q$ implies $\llbracket p \rrbracket \approx \llbracket q \rrbracket$;
- maps rewriting to a silent path: $p \rightsquigarrow q$ implies $\llbracket p \rrbracket (\rightsquigarrow^* \rightsquigarrow) \llbracket q \rrbracket$, and is called **strong** if $\llbracket p \rrbracket \rightsquigarrow^* \rightsquigarrow \llbracket q \rrbracket$;
- and preserves behavior against the transported observers: $p \approx_D^S q$ iff $\llbracket p \rrbracket \approx_{\mathcal{D}} \llbracket q \rrbracket$ in $\llbracket \mathcal{D} \rrbracket$.

The last clause is stated as a biconditional. Soundness — the forward direction — is what a translation must have to be usable at all. The converse is what makes the restriction to $\llbracket \mathcal{D} \rrbracket$ the *right* restriction rather than merely a sufficient one, and it is the direction that fails badly against unrestricted observers, since the target can then separate images of equivalent programs by reading their implementation.

7.2 Eliminating replication

The smallest instance is the elimination of replicated input from $\text{rho}^!$, using a duplicator stored at a name [Lybech, 2022a].

Proposition 38 (Lybech, 2022a). *There is an encoding of $\text{rho}^!$ into rho given by*

$$\begin{aligned} m, n : Nm \quad &\vdash \quad m \cdot n := @(\text{o}(m, 0) \mid \text{i}(n, \lambda x.0)) : Nm \\ x : Nm \quad &\vdash \quad D(x) := \text{i}(x, \lambda y. * y \mid \text{o}(x, *y)) : Pr \\ \llbracket !\text{in}(a, \lambda x.p) \rrbracket &= \lambda n. D(n) \mid \text{out}(n, \text{in}(a, \lambda x. \llbracket p \rrbracket (n \cdot n) \mid D(n))) \end{aligned}$$

The rewrite $!(p) \rightsquigarrow p \mid !(p)$ maps to a path from $\llbracket p \rrbracket$ to a process bisimilar to $\llbracket p \rrbracket \mid !(p)$.

$$\begin{aligned} \llbracket !\text{in}(a, \lambda x.p) \rrbracket (n) &= D(n) \mid \text{out}(n, \text{in}(a, \lambda x. \llbracket p \rrbracket (n \cdot n) \mid D(n))) \\ &= \text{i}(n, \lambda y. * y \mid \text{o}(n, *y)) \mid \text{o}(n, \text{i}(a, \lambda x. \llbracket p \rrbracket \mid D(n))) \\ &\rightsquigarrow *(@(\text{i}(a, \lambda x. \llbracket p \rrbracket \mid D(n)))) \mid \text{o}(n, *(@(\text{i}(a, \lambda x. \llbracket p \rrbracket \mid D(n)))) \\ (1) \quad &\rightsquigarrow \text{i}(a, \lambda x. \llbracket p \rrbracket \mid D(n)) \mid \text{o}(n, *(@(\text{i}(a, \lambda x. \llbracket p \rrbracket \mid D(n)))) \\ (2) \quad &\approx \llbracket \text{in}(a, \lambda x.p) \rrbracket \mid \llbracket !\text{in}(a, \lambda x.p) \rrbracket \end{aligned}$$

A copy of $\llbracket \lambda x.p \rrbracket$ is stored at a and the same is being output; for any receive on a , the body runs and another duplicator opens, reforming the original redex.

$$\begin{aligned} (1) \quad &\xrightarrow{\text{out}(a, r) \mid -} (\llbracket p \rrbracket[@r] \mid D(n)) \mid \text{out}(n, \text{in}(a, \lambda x. \llbracket p \rrbracket \mid D(n))) \\ &= \llbracket p \rrbracket[@r] \mid \llbracket !\text{in}(a, \lambda x.p) \rrbracket \\ (2) \quad &\xrightarrow{\text{out}(a, r) \mid -} \llbracket p \rrbracket[@r] \mid \llbracket !\text{in}(a, \lambda x.p) \rrbracket \end{aligned}$$

Theorem 39. *If $p \approx_{\mathcal{D}} q$ in $\text{T}_{\text{rho}}^!$ then $\llbracket p \rrbracket \approx_{\mathcal{D}} \llbracket q \rrbracket$ in T_{rho} .*

Proof. The only difference of $\text{T}_{\text{rho}}^!$ from T_{rho} is the operation $! : \{\text{in}(n, c)\} \rightarrow Pr$ and the rewrite $!(p) \rightsquigarrow p \mid !(p)$, with no equations between $!$ and the other operations; so it has exactly one redex IPO not already in T_{rho} . It therefore suffices to consider $\lambda x.p \approx_{\mathcal{D}} \lambda x.q$ and apply $\llbracket !\text{in}(n, -) \rrbracket$. Because the result is an abstraction, the first transition must be an evaluation at a particular n .

Suppose $\llbracket !\text{in}(a, \lambda x.p) \rrbracket (n) \xrightarrow{c} t$. Then c is $(-)$, $(- \mid \text{out}(n, q))$, or $(- \mid \text{in}(n, c))$. In the latter two cases the rewrites of the duplicator are mirrored in the two processes. If $c = (-)$ the process evolves to (1) above, and the same transitions apply to $\llbracket !\text{in}(n, \lambda x.q) \rrbracket$; because $\lambda x.p \approx_{\mathcal{D}} \lambda x.q$, they are bisimilar for every substitution $p[@r] \approx_{\mathcal{D}} q[@r]$ by Proposition 21. Hence $\llbracket !\text{in}(a, \lambda x.p) \rrbracket \approx_{\mathcal{D}} \llbracket !\text{in}(a, \lambda x.q) \rrbracket$. $\square$

This is the pattern the rest of the paper generalizes: an operation with a rewrite rule is replaced by a protocol, the rewrite becomes a silent path, and the encoding is behavior-preserving against the encoded observers.

Remark 40 (the restriction is already doing work here). Theorem 39 is $\llbracket \mathcal{D} \rrbracket$ -relative and is false without the restriction, which is easy to miss because the source and target theories are so nearly the same. The duplicator holds the stored process at the parameter n . An arbitrary rho context may send on n — $\text{out}(n, q)$ for any q it likes — whereupon $D(n)$ receives q , runs it, and republishes it in place of the process that was stored. The encoding's store is corrupted and the correspondence with $\text{rho}^!$ is destroyed. No context in $\llbracket \mathcal{D} \rrbracket$ can do this: n is a parameter of the translation and is not the image of any name of $\text{rho}^!$. So even the smallest encoding in this paper needs $\mathcal{D}$.

Remark 41 (input guarding). The restriction of $!$ to input-guarded processes, which §3 obtains from the comprehension of Note 2, is not cosmetic. An encoding of unguarded replication by this duplicator replicates eagerly and therefore diverges; Lybech [2022a] identifies this as a defect of the encoding of Meredith and Radestock [2005] and repairs it by guarding, which is exactly what the subset type $\{p : Pr \mid p = \text{in}(n, c)\}$ enforces here.

7.3 π into rho

The flagship instance of this paper is an encoding of the π calculus into rho. Neither of the two we discuss is proved here, and the history of both is worth stating, because that history is itself evidence for the thesis of this paper: the same restriction on observers was arrived at three times, by different routes, before anyone proposed to make it a component of the construction.

Two errors, and their common cause

Meredith and Radestock [2005] proposed an encoding of the asynchronous choice-free π calculus into rho, parameterized by a pair of names (n, p) , and claimed full abstraction with respect to a name-restricted barbed bisimilarity. Lybech [2022a] showed that the claim fails, in both directions, and diagnosed why.

The translation of parallel composition splits the parameters,

$$\llbracket p \mid q \rrbracket_{n,p} = \llbracket p \rrbracket_{+n,+p} \mid \llbracket q \rrbracket_{n+,p+}$$

while the translation of replication rebinds n and p at runtime, so that they always name the most recently replicated generation. The two mechanisms are incompatible. Substitution in rho does not recur under quotes, so the *statically* incremented names $+n$ and $n+$, which sit inside quotations, cannot be reached by the runtime rebinding. A replicated process therefore loses access to its own name supply, and emits the same name every time where it should emit a fresh one. Lybech [2022a] exhibits two consequences: $!(\nu z)\bar{u}\langle z \rangle \mid Q$ against $(\nu z)!(\bar{u}\langle z \rangle \mid Q)$, which are distinguishable in π and become bisimilar under the translation; and $!(\nu z)\bar{u}\langle z \rangle$ against $!(\nu q)(\nu z)\bar{u}\langle z \rangle$, which are structurally congruent in π and become distinguishable. A third defect is independent: that encoding replicates eagerly, and so always diverges.

The common cause is an invariant on the parameters — *n and p are the most recently replicated names* — that is assumed throughout and never stated, and that the translation itself destroys. This is exactly the failure mode that a criterion of *parameter independence* is designed to catch, and it is why Lybech [2022a] adds one to the Gorla criteria.

Two repairs, and what distinguishes them

There are now two corrections in print or in preparation, and they differ in a way this paper is well placed to describe: not in what they prove about the source language, but in which contexts are allowed to watch.

Lybech [2022a] isolates name generation in a single contextual process reached on a name v that is never updated, so that access to it cannot be lost; n is used for return addresses and replication and is incremented, never rebound. We reproduce his encoding, in the notation of this paper.

Theorem 42 (Lybech, 2022a). *There is an encoding of the π calculus with input-guarded replication into the rho calculus, given as follows.*

$$\begin{array}{ll} n : Nm & \vdash \quad +n := @(\mathbf{o}(n, 0)) : Nm \\ n : Nm & \vdash \quad n+ := @(\mathbf{i}(n, \lambda x.0)) : Nm \\ m, n : Nm & \vdash \quad m \cdot n := @(\mathbf{o}(m, 0) \mid \mathbf{i}(n, \lambda x.0)) : Nm \\ x : Nm & \vdash \quad D(x) := \mathbf{i}(x, \lambda y. *y \mid \mathbf{o}(x, *y)) : Pr \\ x, w, v, s : Nm & \vdash \quad !N(x, w, v, s) := \\ & D(x) \mid \mathbf{o}(x, \mathbf{i}(w, \lambda a. \mathbf{i}(v, \lambda r. D(x) \mid \mathbf{o}(r, *a) \mid \mathbf{o}(w, \mathbf{o}(a, 0)))))) \mid \mathbf{o}(w, *s) \end{array}$$

$$\begin{aligned}
\llbracket p \rrbracket &= \lambda x w v n s. \llbracket p \rrbracket(n, v) \mid !N(x, w, v, s) \\
\llbracket 0 \rrbracket(n, v) &= 0 \\
\llbracket p \mid q \rrbracket(n, v) &= \llbracket p \rrbracket(+n, v) \mid \llbracket q \rrbracket(n+, v) \\
\llbracket \text{out}(a, k) \rrbracket(n, v) &= \text{out}(a, *(k)) \\
\llbracket \text{in}(a, \lambda x. p) \rrbracket(n, v) &= \text{in}(a, \lambda x. \llbracket p \rrbracket(n, v)) \\
\llbracket \nu(\lambda x. p) \rrbracket(n, v) &= \text{out}(v, *(n)) \mid \text{in}(n, \lambda x. \llbracket p \rrbracket(n \cdot n, v)) \\
\llbracket !\text{in}(a, \lambda x. p) \rrbracket(n, v) &= D(n) \mid \text{out}(n, \text{in}(a, \lambda x. D(n) \mid \llbracket p \rrbracket(n \cdot n, v)))
\end{aligned}$$

The name server generates the namespace of left-increments rooted at s ; the parameters x, w, v, s are chosen distinct from the free names of p and outside that namespace.

Note that the parameter v is held fixed by every clause, including parallel composition. That is the fix, and it is the one line of the table that carries it.

The encoding we take as our instance

The second repair [Meredith, 2026a] keeps the intent of the 2005 construction — that there be no runtime source of fresh names — by making every namespace derivation a communication rather than a static increment. Its primitive is a two-step allocator

$$F(\sigma; u, v).p := \text{in}(\sigma, \lambda u. \text{in}(\sigma, \lambda v. p)) \mid \text{o}(\sigma, \text{o}(\sigma, 0)) \mid \text{o}(\sigma, \text{i}(\sigma, \lambda y. 0))$$

which, after two reductions, binds u and v to the two distinct names $L(\sigma) := @(\text{o}(\sigma, 0))$ and $R(\sigma) := @(\text{i}(\sigma, \lambda y. 0))$ — in one order or the other, which is deliberate, since only distinctness is ever needed. Every clause of the translation is an abstraction over an *address*, and plugging is instantiation.

Theorem 43 (Meredith, 2026a). *There is an encoding of the π calculus with input-guarded replication into the rho calculus, with*

$$\begin{aligned}
\llbracket 0 \rrbracket(\sigma) &= 0 \\
\llbracket \text{out}(x, z) \rrbracket(\sigma) &= \text{o}(x, *z) \\
\llbracket \text{in}(x, \lambda y. p) \rrbracket(\sigma) &= \text{i}(x, \lambda y. \llbracket p \rrbracket(\sigma)) \\
\llbracket p \mid q \rrbracket(\sigma) &= \text{i}(\sigma, \lambda m. \llbracket p \rrbracket(m)) \mid \text{i}(\sigma, \lambda m. \llbracket q \rrbracket(m)) \\
&\quad \mid \text{o}(\sigma, \text{o}(\sigma, 0)) \mid \text{o}(\sigma, \text{i}(\sigma, \lambda y. 0)) \\
\llbracket \nu(\lambda z. p) \rrbracket(\sigma) &= F(\sigma; z, m). \llbracket p \rrbracket(m) \\
\llbracket !\text{in}(x, \lambda y. p) \rrbracket(\sigma) &= F(\sigma; a, b). F(b; s, c). (D(a) \mid \text{o}(a, B) \mid \text{o}(s, *c))
\end{aligned}$$

where $B = \text{i}(x, \lambda y. (D(a) \mid \text{i}(s, \lambda c'. F(c'; d, e). (\text{o}(s, *e) \mid \llbracket p \rrbracket(d))))$, and the top-level address is $n_0 := @(\prod_{x \in \text{fn}(p)} \text{o}(x, 0))$. It is fully abstract for a context-labelled bisimilarity whose observers are the encodings $\llbracket C_\pi \rrbracket$ of source contexts:

$$p \approx_D^\pi q \quad \text{iff} \quad \llbracket p \rrbracket \approx_D \llbracket q \rrbracket \quad \text{in} \quad \llbracket C_\pi \rrbracket.$$

Two features of that statement matter here and are worth separating from the encoding that carries them.

Proposition 44 (Meredith, 2026a). *The restriction of observers cannot be dropped. There are π processes $p \equiv q$ and a rho context $c \notin \llbracket C_\pi \rrbracket$ with $c(\llbracket p \rrbracket) \not\approx c(\llbracket q \rrbracket)$; and by Theorem 55 no encoding of π into rho whatever is fully abstract against unrestricted rho observers.*

The witness is $q = p \mid 0$, and it is as small as a counterexample gets: the image of $p \mid 0$ performs the administrative traffic of a parallel node and so has a barb on the address at which that node was invoked, while the image of p has none. The address is computed by the translation from the free names of p , so a rho context can name it; no π context mentions it, so no encoded context can. This is the example of §1.1, and it is the reason we regard $\llbracket \mathcal{D} \rrbracket$ as the shape of the theorem rather than a weakening of it.

Remark 45 (where the restriction is used). In the proof of Theorem 43 the restriction on observers is used exactly once: in the step asserting that two states of one configuration, differing only in how much of the translation’s bookkeeping has been discharged, are indistinguishable. Proposition 44 exhibits an unrestricted context that distinguishes them, and with that step gone the argument fails at its first link. We think this is the most useful diagnostic available for whether a restriction of observers is doing honest work: locate the single place the proof consumes it, and check that an unrestricted observer breaks exactly there.

Remark 46 (the two repairs compared). Both encodings are correct, for the same source language, under their own assumptions, and Lybech’s is at present proved in more detail. They differ in the implementation they presuppose. His posits a freshness oracle on a fixed channel; the name traffic of an image is a star of diameter two. The other posits none, deriving each address from its parent by communication; the name traffic is a tree isomorphic to the parse tree of the source. In the vocabulary of §4 the difference is visible as a difference in $\llbracket \mathcal{D} \rrbracket$: the server channel v is free in the image of every ν , so it is nameable by encoded contexts, whereas an address is nameable by no encoded context except the one that allocated it. We take the second as our running instance because its full-abstraction theorem is already stated for the class of observers this paper constructs.

What is proved, and against which criteria

Lybech [2022a] proves Theorem 42 correct against a list of criteria adapted from Gorla [2010]: compositionality, substitution invariance, operational correspondence, observational correspondence, divergence reflection, and parameter independence. The behavioral equivalence is $\approx^{\mathcal{N}}$, $\mathcal{N}$ -restricted barbed bisimilarity, in which the observation predicate is parameterized by a set of names and only names in that set may be observed; the encoding is judged with $\mathcal{N}$ the free names of the source term, so that names manufactured by the translation are unobservable. Restriction to input-guarded replication is what removes the divergence of the earlier encoding. Meredith [2026a] verifies the same criteria of Theorem 43 as consequences of full abstraction rather than as separate obligations.

Two of Lybech’s results are ones we would otherwise have had to prove here, and we take them as given.

Proposition 47 (Lybech, 2022a, Prop. 1: independence of parameters). *If n, n', s, s' are fresh for p and unique for the translation, then*

$$\llbracket p \rrbracket_{n,v} \mid !N(x, w, v, s) \sim^{\text{fn}(p)} \llbracket p \rrbracket_{n',v} \mid !N(x, w, v, s')$$

Proposition 48 (Lybech, 2022a, Lem. 1: stratification). *Name equivalence implies equality of quote depth, where the quote depth of $@(P)$ is that of x if $P \equiv *x$, and otherwise one more than the quote depth of P . Consequently the namespaces generated by the static increments $+(-)$, $(-)+$ and $(-)\cdot(-)$ from distinct, non-derivable roots are disjoint.*

Proposition 47 is the ancestor of Lemma 63 below, and Proposition 48 is the rigorous form of the name discipline we use in §8. Full proofs are in the accompanying technical report [Lybech, 2022b]. Meredith [2026a] restates parameter independence as *address independence*, a consequence of a relabelling that is a symmetry of the encoded contexts and of no others, and proves it once rather than per clause; that is the form we use in §8.

7.4 Relative minimality, and what transfers

Theorem 43 is stated for a context-labelled bisimilarity whose labels are the *enabling* contexts drawn from $\llbracket C_\pi \rrbracket$. Definition 37 asks for something else: labels given by $\llbracket \mathcal{D} \rrbracket$ -relative IPOs, which are the *minimal* such contexts. The relation between the two is the last thing standing between the theorem and the framework, and it is not symmetric.

Write $\approx_e$ for bisimilarity with all enabling contexts of $\llbracket \mathcal{D} \rrbracket$ as labels, and $\approx_{\mathcal{D}}$ for bisimilarity with $\llbracket \mathcal{D} \rrbracket$ -relative IPO labels, as in §5.

Proposition 49. $\approx_e \subseteq \approx_{\mathcal{D}}$. *Consequently the soundness half of Theorem 43 — that $p \approx_{\mathcal{D}}^{\pi} q$ implies $\llbracket p \rrbracket \approx_{\mathcal{D}} \llbracket q \rrbracket$ — holds for the derived transition system of Definition 18.*

Proof. Every $\llbracket \mathcal{D} \rrbracket$ -relative IPO label is in particular an enabling context in $\llbracket \mathcal{D} \rrbracket$, so the label set of $\approx_{\mathcal{D}}$ is a subset of that of $\approx_e$. Fewer labels means fewer challenges, so any $\approx_e$ -bisimulation is a $\approx_{\mathcal{D}}$ -bisimulation, and the greatest such relations are ordered accordingly. $\square$

The converse does not follow, and it is the direction that carries the content of relative minimality.

Obligation 50 (minimality separates as well as enabling does). Show $\approx_{\mathcal{D}} \subseteq \approx_e$ for $\llbracket \mathcal{D} \rrbracket = \llbracket C_\pi \rrbracket$: that if two encoded terms cannot be separated by a minimal encoded context, they cannot be separated by any encoded context. Equivalently, that every enabling context in $\llbracket C_\pi \rrbracket$ factors, up to $\approx$, through a $\llbracket C_\pi \rrbracket$ -relative IPO for the transition it enables.

Two things are worth saying about Obligation 50. First, it is the sharp form of clause (1) of Obligation 52, and settling it settles that clause. Second, it is a statement about relative pushouts and about nothing else — no property of the encoding enters — which is why it belongs in this paper rather than in [Meredith, 2026a], and why we state it here rather than asking that the companion prove its theorem twice. The reason to expect it is the factorization property that defines a relative IPO: if $c \in \llbracket \mathcal{D} \rrbracket$ enables a transition and c_0 is a $\llbracket \mathcal{D} \rrbracket$ -relative IPO for it, then c factors as $c_1 \circ c_0$ with $c_1 \in \llbracket \mathcal{D} \rrbracket$ by minimality, and the residual c_1 is then simulated by the congruence property of $\approx_{\mathcal{D}}$ (Theorem 23). What is not immediate is that the relative IPO exists for every enabling context, which is Proposition 25, itself a sketch; the two obligations are therefore one piece of work.

Remark 51 (what a larger label set costs). It is worth recording the direction, since it is easy to get backwards. More labels means a finer relation, hence a stronger soundness statement and a weaker completeness statement. Meredith [2026a] deliberately takes the larger label set, which makes his completeness the harder theorem and his soundness the weaker one; Proposition 49 says we inherit the weaker one for free and Obligation 50 says what we must do to inherit the harder one.

7.5 What remains: a comparison, not a construction

The encoding is therefore not an open problem. What is open is the relation between the criteria it satisfies and the notion of encoding of Definition 37.

Obligation 52 (the comparison theorem). Let $\llbracket - \rrbracket$ satisfy the criteria of Lybech [2022a], with $\mathcal{N} = \text{fn}(p)$. Show that $\llbracket - \rrbracket$ is an encoding in the sense of Definition 37, with $\mathcal{D}$ the transported observer subcategory $\llbracket \mathcal{D} \rrbracket$ of Definition 12. Specifically:

1. observational correspondence at $\mathcal{N} = \text{fn}(p)$ implies that $\llbracket \mathcal{D} \rrbracket$ -relative barbs and $\llbracket \mathcal{D} \rrbracket$ -relative IPO labels agree — for which Obligation 50 is the sharp form;
2. operational correspondence implies the silent-path clause $p \rightsquigarrow q \Rightarrow \llbracket p \rrbracket (\rightsquigarrow^* \approx)^* \llbracket q \rrbracket$;
3. substitution invariance and Proposition 47 together imply the equation clause, that $p =_\pi q$ implies $\llbracket p \rrbracket \approx \llbracket q \rrbracket$, for the ν -equations in particular;
4. divergence reflection is what makes the weak relation of Definition 37 non-vacuous.

For Theorem 43 the corresponding statement is shorter, since its equivalence is already context-labelled: only (1) is at issue, and (1) is Obligation 50.

The substance of (1) is that Lybech’s restriction and ours are the same restriction differently presented. He obtains it by parameterizing the barb predicate with a set of *names*; we obtain it by restricting the class of *contexts* to $\llbracket \mathcal{D} \rrbracket$, which §4 derives from an observation relation rather than stipulating. The two should coincide for rho, since a barb on x is a transition under the context $- \mid \text{out}(x, 0)$ or $- \mid \text{in}(x, \lambda y. 0)$, and those contexts are encoded exactly when x is a name of the source; but for a theory whose observations are not exhausted by barbs on names the constructions come apart, and it is the context formulation that generalizes.

Remark 53 (why not barbs). A barbed presentation buys simplicity at the cost of generality: it presumes that the observations worth making are the names on which a term is ready to communicate. That is true of rho and π and false in general — for an arbitrary interactive lambda theory there is no reason for the barbs to be adequate, whereas Definition 17 is available whenever the theory has an observation relation. This is our reason for the context formulation and not a criticism of Lybech [2022a], whose setting is a single pair of calculi where the barbed presentation is exactly right.

Remark 54 (full abstraction). Earlier drafts recorded full abstraction for $\pi \rightarrow \text{rho}$ as an obligation of this paper. It is not one: Theorem 43 is that statement, for the enabling-context presentation, and Proposition 49 together with Obligation 50 is what remains to state it for ours. We note with Gorla and Nestmann [2016] that full abstraction is not by itself the most informative criterion — it does not, for instance, prevent an encoding from introducing divergence — which is why the criteria above are stated separately rather than bundled.

7.6 Separation

The converse fails, and the reason is the subject of this paper.

Theorem 55 (Lybech, 2022a, Thm. 1 and Cor. 1). *There is no encoding of the rho calculus into the π calculus satisfying those criteria, for any behavioral equivalence preserving reductions, observability and substitution. The same holds for $\text{HO}\pi$, by composition with the encoding of Sangiorgi [1993].*

The mechanism is instructive. Let $u := @(*x_1 \mid *x_2)$ and consider

$$P := \text{o}(a, *x_1 \mid *x_2) \mid \text{i}(a, \lambda n. \text{o}(n, 0)) \quad P' := \text{o}(u, 0)$$

Then $P \rightsquigarrow P'$, and u is a *free* name of P' that is not a free name of P . Reflection has manufactured, at runtime, an observable name that no static analysis of the redex could have predicted. In π this is impossible: names have no structure and cannot be composed at runtime, so any free name appearing after a reduction was either free before or came from the translation parameters — and the latter is precisely what parameter independence forbids.

In the vocabulary of §4 this is the same fact as Proposition 10. Choosing $\approx_{Nm}$ to be equality is what makes a manufactured name a genuinely new observation; $@$ is the operation that manufactures one; and $@$ is inadmissible for exactly that reason. The separation result says that a target theory whose observation relation at Nm has no such operation cannot host rho, and the failure of congruence for $\text{out}(@(-), z)$ says that rho cannot admit that operation as an observer of itself. One phenomenon, seen from inside and from outside.

Obligation 56 (separation, $\mathcal{D}$ -relatively). Re-derive Theorem 55 from Definition 37: show that no functor $\text{rho} \rightarrow \llbracket \mathcal{D} \rrbracket_\pi / \approx$ exists, using the inadmissibility of $@$ rather than a criteria list. If the framework of §4 is doing real work, this is where it should show, since the argument ought to become a statement about which operations can appear in an observation relation rather than a case analysis on substitutions.

A route is now visible, in the argument Meredith [2026a] uses to show that his restriction is forced. Encoded contexts use a received name only as the subject of a communication or as the object of a lift; they never drop it. Dropping is the operation $*$, and $*(@(-))$ is admissible while $@$ alone is not, so the asymmetry is already recorded in §4: the operation that reads a name’s structure is available to arbitrary rho contexts and to no encoded one. What is wanted is the observation that a translation into a theory whose observation relation at Nm is generated without such an operation cannot have this asymmetry to exploit, and therefore cannot be faithful — which is Theorem 55 with the case analysis replaced by a statement about the generators of $\approx_{Nm}$.

We regard Obligation 56 as the sharpest available test of this paper’s apparatus. A framework that reorganizes known positive results is worth something; one that also reproves a separation result more structurally is worth considerably more.

8 Third instance: extension by data types

We come to the motivating case. A user of MeTTaIL presents a pure theory — λ , rho, π , or a domain-specific calculus of their own — with no built-in data. They then want **Bool**, **Int**, **String**, lists. Adding these adds operations and, crucially, *equations*: $2 + 3 = 5$ is not a rewrite the user writes, it is an identity they expect to hold. The question is whether the reasoning they do in the extended theory survives compilation back into the pure one.

8.1 Extensions and models

Definition 57. A *finitary algebraic theory* A is a presentation with finitely many sorts, finitely many operations of finite arity, and equations; no rewrites. Its **initial model** is the free model on no generators.

Definition 58. Let T be a lambda theory and A a finitary algebraic theory. The **extension** $T \otimes A$ is T together with the sorts, operations and equations of A and an injection $\iota_a : a \rightarrow Pr$ for each sort a of A . The extension is **pure** if no new equation mentions an operation of T other than the injections, and **conservative** if $T \rightarrow T \otimes A$ reflects equations and rewrites between terms of T .

Purity and conservativity are exactly the conditions a language-definition platform can check mechanically from a presentation, and they are what make the following possible; without purity there is no reason to expect an encoding that is the identity on T .

Definition 59. A **lax model** M of A in T assigns to each sort a an object $M(a)$ of T and to each operation f of A a morphism $M(f)$ of the corresponding type, such that every equation of A holds up to weak $\mathcal{D}$ -bisimilarity:

$$A \vdash s = t \quad \text{implies} \quad M(s) \approx_{\mathcal{D}} M(t)$$

The word *lax* is the only honest one. A strict model would satisfy the equations on the nose, and no encoding of a data type into a process calculus does: the built-in $2 + 3$ is an identity, taking no steps, while its encoding is a protocol taking several. Laxity is not a weakness of the construction; it is what the construction is about.

8.2 Canonicity

There are many encodings of **Bool** into a calculus — Church, Scott, Parigot, and any number of ad hoc ones. Calling one *canonical* requires saying with respect to what.

Proposition 60. Let A be finitary algebraic and M a lax model of A in T . Then M determines, up to weak $\mathcal{D}$ -bisimilarity, a unique map on the A -terms of $T \otimes A$ commuting with the operations. Canonicity is thus canonicity relative to the choice of M , and the encoding of the closed A -terms is determined by initiality.

Proof. The closed A-terms form the initial model of A; a lax model is a model in the quotient $\mathcal{D}/\approx$ by Proposition 32, so there is a unique morphism from the initial model to it, and uniqueness in the quotient is uniqueness up to $\approx$. This is the free-extension machinery of Proposition 80 applied to the inclusion of the empty presentation. $\square$

For the calculi of §3 we take M to be the **Scott-style** model, in which a value of an algebraic sort is a process that, when handed a family of continuation names, signals on the one corresponding to its outermost constructor and delivers its arguments there. In a calculus with pattern matching this is the right choice: destructuring is one communication rather than a fold, which matters for a platform whose surface language has a for-comprehension with patterns. Church-style models are also lax models and everything below applies to them; they are simply worse.

8.3 The name discipline

Scott-style encodings need a supply of private names. In rho these are generated by quotation, using the operations already introduced in §7:

$$+n := @(\mathbf{o}(n, 0)) \quad n+ := @(\mathbf{i}(n, \lambda x.0)) \quad n \cdot n := @(\mathbf{o}(n, 0) \mid \mathbf{i}(n, \lambda x.0))$$

Every encoded term is parameterized by a root name n , and the parallel clause splits the root, $\llbracket p \mid q \rrbracket(n) = \llbracket p \rrbracket(+n) \mid \llbracket q \rrbracket(n+)$, so that occurrences receive distinct roots.

Lemma 61 (name discipline). *Under the root-splitting discipline, distinct occurrences of encoded subterms receive incomparable roots, and the derived names of one occurrence are the derived names of no other.*

Proof. The maps $n \mapsto +n$ and $n \mapsto n+$ are injective with disjoint images, since $\mathbf{o}(n, 0)$ and $\mathbf{i}(n, \lambda x.0)$ are distinct terms and $@$ is injective. So roots form a binary tree and the derived names of a node lie in its own subtree. The general statement, for arbitrary namespaces generated by static quoting, is Proposition 48: name equivalence preserves quote depth, so namespaces built by the same template from non-derivable roots are disjoint. Meredith [2026a] states the same fact for the templates L and R of Theorem 43 and decomposes it into its two ingredients — stratification by quote depth separates namespaces of different depth, template disjointness separates siblings — which is the form to cite when the roots are generated at runtime rather than written down. $\square$

Remark 62 (a failure mode this discipline does not by itself exclude). Static increments sit inside quotations, and substitution in rho does not recur under quotes. A root parameter that is *bound* somewhere in an encoding and re-substituted at runtime will therefore not reach the increments derived from it, and the encoding will reuse names it believes to be fresh. This is precisely the error Lybech [2022a] found in Meredith and Radestock [2005], described in §7.3. The encoding of §8.4 is safe because no root is ever bound: the conditional is not replicated and n is threaded statically. An encoding of a recursive type whose fold replicates its continuation would not be safe for free, and the discipline must be checked, not assumed, for each model M.

Lemma 63 (root independence; cf. Lybech, 2022a, Prop. 1). *For m, n roots not occurring in $\llbracket p \rrbracket$, $\llbracket p \rrbracket(m) \approx_{\mathcal{D}} \llbracket p \rrbracket(n)$ in $\llbracket \mathcal{D} \rrbracket$.*

Proof. The root parameter appears only in derived names used for internal signalling. By Lemma 61 those names lie in the occurrence’s own subtree, and by Definition 12 no encoded context addresses them, since the source theory has no operation whose image sends on a derived name of another occurrence. So the relation pairing $\llbracket p \rrbracket(m)$ with $\llbracket p \rrbracket(n)$ and closed under the transitions of $\llbracket \mathcal{D} \rrbracket$ is a $\mathcal{D}$ -bisimulation.

The argument is worth having in its sharper form, which Meredith [2026a] supplies and which we adopt rather than reprove. The bijection $\beta_{m \rightarrow n}$ relabelling one root’s namespace to another’s is *not* a substitution of the theory — it must descend under quotes, and rho substitution does

not — so it is available to the metatheory and to no context whatever. That it is nonetheless a bisimulation for $\llbracket \mathcal{D} \rrbracket$ is exactly his name-usage lemma: an encoded context uses a received name only as a subject or as a transmitted value, so the only discrimination it can make between two names is an equality test, which a bijection preserves and reflects. $\square$

Lemma 63 is false against unrestricted rho observers, which can send on $+m$ and not on $+n$. It is the first place where the whole apparatus of §4 earns its keep, and it is not a technicality: without it no encoding of a binder-free data type is well defined.

8.4 Booleans, in full

Let A_{Boo1} have one sort, constants $\texttt{tt}, \texttt{ff}$, an operation $\texttt{if}(-, -, -)$, and the two equations

$$\texttt{if}(\texttt{tt}, x, y) = x \quad \texttt{if}(\texttt{ff}, x, y) = y$$

from which negation, conjunction and disjunction are definable in the usual way. The Scott model in rho is

$$\begin{aligned} \llbracket \texttt{tt} \rrbracket(n) &= \texttt{out}(+n, 0) \\ \llbracket \texttt{ff} \rrbracket(n) &= \texttt{out}(n+, 0) \\ \llbracket \texttt{if}(b, p, q) \rrbracket(n) &= \llbracket b \rrbracket(n) \mid \texttt{in}(+n, \lambda y. \llbracket p \rrbracket(n \cdot n)) \mid \texttt{in}(n+, \lambda y. \llbracket q \rrbracket(n \cdot n)) \end{aligned}$$

Lemma 64 (dead branches are inert). *Let n be the root of a conditional whose test has been consumed. Then $\texttt{in}(n+, \lambda y. \llbracket q \rrbracket(n \cdot n)) \approx_{\mathcal{D}} 0$ in $\llbracket \mathcal{D} \rrbracket$.*

Proof. A transition out of the residual requires an observation sending on $n+$. By Lemma 61 the only encoded producer of a message on $n+$ is $\llbracket \texttt{ff} \rrbracket(n)$, and the root n belongs to this occurrence alone; the test has been consumed, so no such producer remains in the term. By Definition 12 no context in $\llbracket \mathcal{D} \rrbracket$ supplies one either, since every encoded context addresses names derived from its own root. Hence the residual has no transitions in the $\llbracket \mathcal{D} \rrbracket$ -relative system, and the singleton relation pairing it with 0 is a $\mathcal{D}$ -bisimulation. $\square$

Proposition 65. *The Scott model of A_{Boo1} in rho is a lax model: both equations become weak $\llbracket \mathcal{D} \rrbracket$ -bisimulations.*

Proof. Compute the first; the second is symmetric.

$$\begin{aligned} \llbracket \texttt{if}(\texttt{tt}, p, q) \rrbracket(n) &= \texttt{out}(+n, 0) \mid \texttt{in}(+n, \lambda y. \llbracket p \rrbracket(n \cdot n)) \mid \texttt{in}(n+, \lambda y. \llbracket q \rrbracket(n \cdot n)) \\ &\rightsquigarrow \llbracket p \rrbracket(n \cdot n) \mid \texttt{in}(n+, \lambda y. \llbracket q \rrbracket(n \cdot n)) \\ &\approx_{\mathcal{D}} \llbracket p \rrbracket(n \cdot n) \quad \text{by Lemma 64} \\ &\approx_{\mathcal{D}} \llbracket p \rrbracket(n) \quad \text{by Lemma 63} \end{aligned}$$

The first step is the communication rewrite; it is a silent step, so the composite is a weak bisimulation and not a strong one. $\square$

Remark 66. Every line of that computation after the first is $\llbracket \mathcal{D} \rrbracket$ -relative and false without the restriction. Against arbitrary rho observers the residual $\texttt{in}(n+, \dots)$ is live — a context may send on $n+$ and fire the branch that was not taken — so $\llbracket \texttt{if}(\texttt{tt}, p, q) \rrbracket$ and $\llbracket p \rrbracket$ are separated, and the compiled conditional is simply wrong. This is the clearest statement we know of why the observer subcategory cannot be dispensed with. It is also the standard “junk” problem of encodings, and the $\mathcal{D}$ -relative treatment is the standard fix, made into a definition.

8.5 The general theorem

Theorem 67. *SKETCH Let T be a lambda theory with all $\mathcal{D}$ -redex-RPOs, A a finitary algebraic theory, $T \otimes A$ a pure conservative extension, and M a lax model of A in T whose operations are admissible. Then the induced $\llbracket - \rrbracket_M : T \otimes A \rightarrow T$ is an encoding in the sense of Definition 37: it is the identity on T , and every equation of $T \otimes A$ becomes a weak $\mathcal{D}$ -bisimulation.*

Proof sketch. Identity on T is purity: no new equation constrains an old operation, so the old fragment is untouched, and conservativity ensures nothing new is derivable there.

For the equations, induct on the derivation of $s = t$ in $T \otimes A$. The axioms of A hold up to $\approx_{\mathcal{D}}$ by Definition 59. Reflexivity, symmetry and transitivity are inherited because $\approx_{\mathcal{D}}$ is an equivalence. Congruence steps — from $s_i = t_i$ conclude $f(\vec{s}) = f(\vec{t})$ — hold because the operations of M are admissible, so $\approx_{\mathcal{D}}$ is a congruence for them by Proposition 7, or in the case of ρ by Theorem 34. Substitution steps hold by Proposition 21. Purity is what guarantees that no derivation mixes an A -axiom with a T -equation in a way not covered by these cases.

Two gaps. First, the induction is over derivations in the free infinitary theory, and the infinitary conjunction defining $\approx_{\mathcal{D}}$ must be shown to commute with the induction; this is where a well-foundedness hypothesis on A is needed and A being finitary is what should supply it. Second, admissibility of the operations of M is a per-model check, and we have carried it out only for A_{Bool} . $\square$

Obligation 68. Strengthen Theorem 67 to a retraction: exhibit $T \otimes A \rightarrow T$ and $T \rightarrow T \otimes A$ with the composite $\llbracket \mathcal{D} \rrbracket$ -relatively bisimilar to the identity, naturally in T . Naturality in T is the formal content of the claim that extensions “just work” for any pure theory a user may present, rather than for each one separately.

8.6 What is out of scope, and why

Remark 69 (Float). We scope the theorem to initial algebras of finitary signatures: `Bool`, `Nat` and `Int`, `String` as a free monoid, lists, finite maps. Floating-point arithmetic is not among them and should not be. IEEE 754 addition is not associative, has two zeros, and has a value unequal to itself; there is no algebraic theory whose initial model it is, hence nothing for an encoding to be canonical with respect to. The right treatment is to carry `Float` as an opaque ground type with an oracle, whose observation relation at that type is a chosen equivalence in the sense of Definition 5 — which is precisely the freedom that definition leaves open at base types other than *Pr*.

Remark 70 (cost). In a cost-accounted theory Theorem 67 is false as stated. The built-in $2 + 3$ and the encoded $\llbracket 2 + 3 \rrbracket$ carry different ledgers: the first is an equation and costs nothing, the second is a protocol and costs several communications. No cost-respecting bisimulation relates them.

We regard the gap as the content rather than an obstruction. The encoding theorem says that extension by data types is *semantically* free; cost accounting says that data types are exactly what one pays less for. Together they say why a Turing-complete language acquires an `Int` at all. The two statements should be proved separately, over the uncosted and costed theories respectively, and their difference — the ledger gap of an encoding — is a quantity worth naming.

Remark 71 (what a platform must check). Theorem 67 is stated so that its hypotheses are decidable properties of a presentation. Given a MeTTaIL presentation of T and of an extension, a compiler can check that the extension is pure (no new equation mentions an old operation), that A is finitary algebraic (no rewrites among the new rules), and that the supplied model’s operations are admissible for the declared observation relation. Conservativity and the well-foundedness gap of Theorem 67 are the parts that are not obviously mechanical, and are where we would look first if the guarantee is to be delivered by a tool rather than by hand.

9 Related work

Deriving labels. The framework is that of Leifer and Milner [2000], refined for bigraphs by Milner and Jensen [2004]. Klin et al. [2005] give a general account of the “labels from reductions” programme and its limits. Our departure is to make the observer class a parameter rather than take it to be the ambient category; the technique of restricting to a subcategory is theirs, applied to reaction rather than observation.

Structural congruence and 2-categories. Strict relative pushouts are fragile in the presence of equations such as associativity and commutativity of parallel composition. Sassone and Sobociński [2003, 2005] replace them with groupoidal relative pushouts in a bicategorical setting. As noted in Remark 30, our normality assumption addresses the same difficulty by a different route and the relationship deserves to be settled rather than left implicit; our matching of labels up to $\approx$ is already partway there.

Higher-order syntax. Presenting binding algebraically goes back to Fiore et al. [1999]; Arkor and McDermott [2021] give the higher-order algebraic theories closest in spirit to our lambda theories. What we add is the treatment of rewriting as a proposition in the subobject fibration, so that types, terms, equations and rewrites are one structure and entailments may be carried to reason under binders. Di Gianantonio et al. [2009] apply RPOs to the λ -calculus using second-order contexts and supply the IPO-uniformity condition on which Theorem 24 is modelled.

Encodings. Gorla [2010] surveys the criteria by which an encoding between process calculi is judged — compositionality, name invariance, operational correspondence, divergence reflection, success sensitiveness — and van Glabbeek [2018] derives a notion of valid encoding from a semantic equivalence instead of a list. Gorla and Nestmann [2016] assess what full abstraction does and does not deliver. Definition 37 is closer to van Glabbeek’s programme than to Gorla’s: one condition, that the translation be a functor into $\llbracket \mathcal{D} \rrbracket / \approx$, from which criteria should follow. We think this matters beyond economy. A list of criteria has to be argued for case by case, and compositionality in particular imports a syntactic constraint that bisimulation preservation was meant to escape: Milner’s encoding of λ into π is behaviorally faithful without being a homomorphism of signatures. Our position is that such constraints belong downstream, as refinements of a behavioral condition, rather than as primitives alongside it. Establishing that, for the rho calculus at least, is Obligation 52.

The rho calculus in particular. Milner [1992] is the origin of the question for λ into π and Boudol [1997] gives the name-passing variant that makes the translation tractable. Meredith and Radestock [2005] proposed the first π -into-rho translation; Lybech [2022a] identified two errors in it, gave the corrected encoding we reproduce in §7.3, and proved the separation result of §7.6, with full proofs in [Lybech, 2022b]. Bendixen et al. [2022] instantiate the rho calculus as a higher-order Ψ -calculus, which is a third framework in which these questions can be posed. Scott-style encodings of data are folklore; Mogensen [1992] is the usual reference.

Ambients. Bonchi et al. [2009] treat the ambient calculus in the reactive-systems framework and find that the IPO semantics must be coarsened by barbs. We take this as evidence that a locality operator needs a different construction, and omit ambients accordingly (Remark 4).

10 A proposal: morphisms should carry contexts

We close with a proposal about the category in which all of this should live. It is not a theorem, and we state it as a proposal rather than dress it as one; but it is the shape the preceding sections keep pointing at, and we think it is the right way to organize the next round of work.

10.1 The received definition is incomplete

The semantic category of graph-structured lambda theories has been given, in companion work [Meredith, 2026b], with theories as objects and *bisimulation-preserving maps* as morphisms, where

the bisimulation is the one over context-labelled transitions and is required to be a congruence.

That definition does not quite parse. The labels of a context-labelled transition system are contexts. So a map that preserves context-labelled bisimulation must already say what it does to labels, which is to say what it does to *contexts* — and a map of terms does not determine one. Two encodings agreeing on every term may disagree on how a source context is realized in the target, and they will then preserve different relations. The received definition quantifies over a datum it never supplies.

Our proposal is to supply it. This is not a strengthening of the morphism class; it is the completion of a definition that was underdetermined.

10.2 The proposal

Definition 72. Let T be a lambda theory. Its **context multicategory** $C(T)$ has the types of T as objects, and as multimorphisms $c : (A_1, \dots, A_n) \rightarrow B$ the n -hole contexts of T , with composition given by plugging. The unary contexts $Pr \rightarrow Pr$ are the labels of the derived transition system of §5, and the rewrite constructors form a sub-multicategory.

Definition 73. A **morphism** of graph-structured lambda theories $S \rightarrow T$ is a pair (F, Φ) where F maps terms of S to terms of T and $\Phi : C(S) \rightarrow C(T)$ is a multifunctor on contexts, subject to

$$\begin{aligned} \text{equivariance} \quad & F(c[t_1, \dots, t_n]) \approx \Phi(c)[Ft_1, \dots, Ft_n] \\ \text{preservation} \quad & t \xrightarrow{c}_S u \implies Ft \xrightarrow{\Phi(c)}_{T*} \approx Fu \end{aligned}$$

A morphism is an **embedding** if in addition Φ is faithful and preservation is reflected: $Ft \xrightarrow{\Phi(c)} z$ implies $t \xrightarrow{c}_S u$ for some u with $z \approx Fu$.

Three things follow immediately, and they are the reason for the proposal.

Proposition 74. For a morphism (F, Φ) , the transported observer subcategory of Definition 12 is the image of Φ . Consequently $[[\mathcal{D}]]$ -relative bisimilarity is bisimilarity relative to $\text{im } \Phi$, and the relative minimality of §5 is minimality in the image of Φ .

The observer subcategory therefore stops being a parameter one supplies alongside a translation and becomes a component of the translation itself. Everything $\mathcal{D}$ -relative in this paper is, on this reading, relative to the context component of a morphism.

Proposition 75. An encoding in the sense of Definition 37 is precisely a morphism $S \rightarrow T$ in the sense of Definition 73 whose equivariance square commutes in $\text{im } \Phi / \approx$ rather than on the nose.

So the paper’s notion of encoding and the category’s notion of morphism, which have been maintained separately, are one notion. The clause of Definition 37 demanding that source equations become bisimulations is not an extra condition on encodings; it is what equivariance up to $\approx$ says.

Proposition 76. Under Definition 73, the hosting and exhausting conditions of [Meredith, 2026b] are properties of Φ rather than of F : hosting is faithfulness of Φ , and exhausting is density, in the sense that every context of the target is $\approx$ -equivalent to one in the image.

That relocation is worth something on its own. Stated of the term map, hosting and exhausting look like bespoke criteria in the style of Gorla [2010]; stated of Φ , they are the two standard properties one asks of a functor.

Conjecture 77. The assignment of a spatial-behavioural logic to a theory is functorial on morphisms in the sense of Definition 73, though not on bisimulation-preserving term maps alone.

The reason to expect this is that the structural connectives of such a logic are indexed by term constructors, and a term constructor is a context; so Φ transports the structural layer, which a term map has no means of doing. Companion work observes that the structural layer strictly refines context-labelled bisimulation and concludes that the assignment fails to be functorial on the received morphism class. Conjecture 77 says the failure was an artifact of the morphisms, not of the logic.

10.3 Four things to settle first

We do not think Definition 73 is finished, and we would rather name the open choices than present it as though it were.

1. **Multifunctor or profunctor?** Definition 73 asks each source context for a single target context. An encoding offering several realizations of one source context — differing in scheduling, or in which of several equivalent name supplies is used — is not obviously pathological, and would want Φ relational. The question is whether any encoding we care about needs this.
2. **Strict or up to $\approx$, and hence bicategorical?** Proposition 75 already forces the equivariance square to commute up to $\approx$, since the ν -equations of π survive translation only as bisimilarities. Taking $\approx$ as a family of 2-cells makes the whole structure a bicategory. Against this, one datum: for the encoding of Theorem 43 the equivariance square commutes on the nose, since $\llbracket c(p) \rrbracket = \llbracket c \rrbracket(\llbracket p \rrbracket)$ literally [Meredith, 2026a]. The strict case is therefore not vacuous, and the laxness is demanded by particular examples — the ν -equations — rather than by the shape of the definition, which is an argument for making strictness the default and laxness the decorated case rather than the other way round. We note also that this merges with a decision already open at Remark 30: strict relative pushouts are fragile under the associativity and commutativity of parallel composition, and the standard repair [Sassone and Sobociński, 2003, 2005] is likewise bicategorical. Two problems, one decision, and their agreement is evidence about which way to take it.
3. **Faithfulness is a property, not part of morphism-hood.** It is tempting to require Φ faithful in Definition 73, since that is what makes an encoding trustworthy. We think this would be a mistake. Building full abstraction into the definition leaves too few morphisms for the category to support the free-forgetful adjunctions of the cost and history monads, and excludes maps one wants for other reasons — the inclusion $T \rightarrow T \otimes A$ of §8 among them. Define morphisms minimally; let faithfulness pick out the embeddings.
4. **Do the decorations lift?** The cost and history monads are defined over the received morphism class. Cost decorations sit on rewrite edges, and a context map must be shown to respect them before the adjunctions can be restated. We expect no difficulty and have not checked.

10.4 What this asks of the companion documents

If Definition 73 is adopted, the division of labour between this paper and the reference account of graph-structured lambda theories becomes clear, where at present the two overlap. The machinery and its justification — observation relations, admissible observers, relative pushouts, and the encoding results that motivate them — belong here. The category, its objects, and the monads over it belong there, and should cite this paper for the morphisms rather than restate them. What is currently duplication becomes a dependency.

We also expect the proposal to change what the reference account can claim. A category whose morphisms carry contexts is one in which the generated logics may be functorial (Conjecture 77), in which relative notions of expressiveness are properties of a single arrow rather than of a translation

plus a side condition, and in which the question this paper began with — which contexts may legitimately observe a program — is answered by the arrow itself.

11 Conclusion and open problems

We have made the class of admissible observers a parameter of the Leifer–Milner construction, shown that the parameter is determined by a choice of equivalence at the ground types, and given a congruence theorem whose statement is $\mathcal{D}$ -relative on both sides. Three phenomena that look unrelated — reflection in the rho calculus, the restriction of an encoding’s observers to encoded contexts, and the correctness of compiling away built-in data — are the same theorem with three choices of $\mathcal{D}$.

The claim that a framework of this kind earns its keep is best tested against a theorem someone needed for other reasons. Theorem 43 is such a theorem: an encoding of π into rho, proved fully abstract, whose statement quantifies over the encodings of source contexts and cannot be made to quantify over more (Proposition 44). It was not written to vindicate the apparatus of §4, and it uses the restriction exactly once, at the one point where the encoding’s own bookkeeping would otherwise become visible (Remark 45). We regard that as better evidence for the parameterization than any number of recovered facts.

Section 10 then argues that $\mathcal{D}$ ought not to be a parameter at all. If a morphism of theories carries a map on contexts as well as on terms — which a context-labelled bisimulation already presupposes, and which the received definition never supplies — then $\mathcal{D}$ is the image of that map, the notion of encoding used throughout this paper coincides with the notion of morphism, and the conditions one imposes on encodings become the standard properties of a functor. We regard this as the paper’s main structural suggestion, and the reason the preceding sections were worth writing in the order they were: the restriction on observers was discovered three times as an obstacle before it could be recognized as a component.

The outstanding work, in the order we would attack it:

1. **Minimality does not lose separating power** (Obligation 50), together with the existence of relative IPOs (Proposition 25) on which it depends. This is now the single most valuable outstanding item: it is what turns the proved full abstraction of Theorem 43 into a theorem about Definition 37, it is a statement about relative pushouts and about nothing else, and Proposition 49 already gives half of what is needed.
2. **Decomposition-closure** (Obligation 26). Whether $\llbracket C_\pi \rrbracket$ is closed under decomposition up to $\approx$ inside rho. Cheap to test, and cheaper for Theorem 43 than for Theorem 42, since there the translation of a context is a context on the nose.
3. **The comparison theorem** (Obligation 52). Earlier drafts of this material treated the correctness of $\pi \rightarrow$ rho as an open problem. It is not: Lybech [2022a] settled it against Gorla-style criteria, and Meredith [2026a] settled it again, differently, against a context-labelled equivalence. What is open is whether a criteria list implies Definition 37, which is a smaller and better-posed question, and for the second encoding it reduces to the item above.
4. **Separation, derived rather than checked** (Obligation 56). The sharpest test of the apparatus.
5. **Weak congruence** (Theorem 24). Every result in §7 and §8 is about weak bisimilarity, so the general statement is load-bearing; at present only the direct argument for rho is available.
6. **Existence of relative IPOs** (Proposition 25). The codirectedness argument needs a well-foundedness condition we have not identified.

7. **Strict versus groupoidal pushouts** (Remark 30). Either justify normality as a checkable hypothesis on presentations, or redo the construction with GRPOs.
8. **The retraction** (Obligation 68), and with it `Int`, `String` and lists, which we expect to follow the pattern of §8.4 without new ideas.
9. **Cost**. Remark 70 identifies a quantity — the ledger gap of an encoding — that measures what a built-in operation buys. We have not studied it.
10. **The morphism class** (§10). Four choices to settle, of which the second — whether to go bicategorical — is the same choice as item 6 above and should be made once.

References

- J. Adámek, J. Rosický, and E. M. Vitale. *Algebraic Theories: A Categorical Introduction to General Algebra*. Cambridge Tracts in Mathematics. Cambridge University Press, 2010. URL https://perso.uclouvain.be/enrico.vitale/gab_CUP2.pdf.
- Nathanael Arkor and Dylan McDermott. Higher-order algebraic theories. 2021. URL <https://arkor.co/files/Higher-order%20algebraic%20theories.pdf>.
- M. Barr and C. Wells. *Toposes, Triples and Theories*. Grundlehren der mathematischen Wissenschaften. Springer New York, 2013. ISBN 9781489900210. URL <https://cjhb.site/Files.php/Books/%28Uncategorized%29/Before%202022/Toposes,%20Triples%20and%20Theories%20-%20M.%20Barr,%20W.%20Wells.pdf>.
- Alex Rønning Bendixen, Bjarke Bredow Bojesen, Hans Hüttel, and Stian Lybech. A generic type system for higher-order ψ -calculi. In *Combined Workshop on Expressiveness in Concurrency and Structural Operational Semantics (EXPRESS/SOS)*, volume 368 of *EPTCS*, pages 43–59, 2022. doi: 10.4204/EPTCS.368.3.
- Filippo Bonchi, Fabio Gadducci, and Giacomina Valentina Monreale. Reactive systems, barbed semantics, and the mobile ambients. volume 5504 of *Lecture Notes in Computer Science*, pages 272–287. Springer, 2009. doi: 10.1007/978-3-642-00596-1_20. URL https://link.springer.com/content/pdf/10.1007/978-3-642-00596-1_20.pdf.
- Gérard Boudol. The pi-calculus in direct style. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '97, page 228–242, New York, NY, USA, 1997. Association for Computing Machinery. ISBN 0897918533. URL <https://doi.org/10.1145/263699.263726>.
- Pietro Di Gianantonio, Furio Honsell, and Marina Lenisa. Rpo, second-order contexts, and lambda-calculus. *Logical Methods in Computer Science*, Volume 5, Issue 3, August 2009. ISSN 1860-5974. URL <https://arxiv.org/abs/0906.2727>.
- Marcelo Fiore, Gordon Plotkin, and Daniele Turi. Abstract syntax and variable binding. In *Logic in Computer Science (LICS)*, pages 193–202, 1999.
- Daniele Gorla. Towards a unified approach to encodability and separation results for process calculi. *Information and Computation*, 208(9):1031–1053, 2010.
- Daniele Gorla and Uwe Nestmann. Full abstraction for expressiveness: history, myths and facts. *Mathematical Structures in Computer Science*, 26(4):639–654, 2016. doi: 10.1017/S0960129514000279.
- B. Jacobs. *Categorical Logic and Type Theory*. Number 141 in Studies in Logic and the Foundations of Mathematics. North Holland, Amsterdam, 1999. URL <https://annas-archive.org/search?q=jacobs+categorical+logic>.
- Peter T Johnstone. *Sketches of an elephant*. Oxford logic guides. Oxford Univ. Press, New York, NY, 2002. URL <https://cds.cern.ch/record/592033>.
- Bartek Klin, Vladimiro Sassone, and Paweł Sobociński. Labels from reductions: Towards a general theory. In *Algebra and Coalgebra in Computer Science*, pages 30–50. Springer Berlin Heidelberg, 2005. ISBN 978-3-540-31876-7. URL <https://www.cs.ox.ac.uk/people/bartek.klin/papers/calco05B.pdf>.
- Anders Kock. Strong functors and monoidal monads. *Archiv der Mathematik*, 23:113–120, 12 1972. doi: 10.1007/BF01304852.

- J. Lambek and P.J. Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 1988. ISBN 9780521356534. URL https://books.google.com/books?id=6PY_emBeGjUC.
- James J. Leifer and Robin Milner. Deriving bisimulation congruences for reactive systems. In Catuscia Palamidessi, editor, *CONCUR 2000 — Concurrency Theory*, pages 243–258, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg. ISBN 978-3-540-44618-7. URL https://www.pure.ed.ac.uk/ws/portalfiles/portal/17894421/Leifer_and_Milner_2000_Deriving_Bisimulation_Congruences.pdf.
- Stian Lybech. Encodability and separation for a reflective higher-order calculus. In *Combined Workshop on Expressiveness in Concurrency and Structural Operational Semantics (EXPRESS/SOS)*, volume 368 of *EPTCS*, pages 95–112, 2022a. doi: 10.4204/EPTCS.368.6. URL <https://arxiv.org/abs/2209.02356>.
- Stian Lybech. Encodability and separation for a reflective higher-order calculus. Technical report, Department of Computer Science, Reykjavík University, 2022b. URL http://icetcs.ru.is/stian/2022/reflection_encodability2022techreport.pdf.
- L. G. Meredith. Namespaces without a name server: Freshness by communication, and a fully abstract encoding of the π -calculus in the rho calculus. F1R3FLY.io, in preparation, 2026a.
- L. G. Meredith. Probing universality: Hosting and exhausting for graph-structured lambda theories. F1R3FLY.io, working note, in preparation, 2026b.
- L.G. Meredith and Matthias Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005. ISSN 1571-0661. doi: <https://doi.org/10.1016/j.entcs.2005.05.016>. URL <https://www.sciencedirect.com/science/article/pii/S1571066105051893>. Proceedings of the Workshop on the Foundations of Interactive Computation (FInCo 2005).
- R. Milner. *A Calculus of Communicating Systems*. Springer-Verlag, Berlin, Heidelberg, 1982. ISBN 0387102353. URL <https://www.lfcs.inf.ed.ac.uk/reports/86/ECS-LFCS-86-7/ECS-LFCS-86-7.pdf>.
- Robin Milner. Functions as processes. *Mathematical Structures in Computer Science*, 2(2):119–141, 1992.
- Robin Milner. The polyadic pi-calculus: a tutorial. 1993. URL <https://api.semanticscholar.org/CorpusID:11201988>.
- Robin Milner and Ole Høgh Jensen. Bigraphs and mobile processes (revised): Technical report 580. Workingpaper, February 2004. URL <https://www.cl.cam.ac.uk/archive/rm135/Jensen-monograph.pdf>.
- Torben Æ. Mogensen. Efficient self-interpretation in lambda calculus. *Journal of Functional Programming*, 2(3):345–363, 1992.
- Gordon D. Plotkin. A structural approach to operational semantics. In *J. Log. Algebr. Program.*, volume 60-61, pages 17–139, 2004.
- Davide Sangiorgi. From π -calculus to higher-order π -calculus — and back. In *TAPSOFT’93: Theory and Practice of Software Development*, pages 151–166, 1993. doi: 10.1007/3-540-56610-4_62.
- Vladimiro Sassone and Paweł Sobociński. Deriving bisimulation congruences using 2-categories. *Nordic Journal of Computing*, 10(2):163–183, 2003.

Vladimiro Sassone and Paweł Sobociński. Locating reaction with 2-categories. *Theoretical Computer Science*, 333(1-2):297–327, 2005.

Rob van Glabbeek. A theory of encodings and expressiveness. In *Foundations of Software Science and Computation Structures (FoSSaCS)*, pages 183–202, 2018. doi: 10.1007/978-3-319-89366-2_10.

A Inference rules for lambda theories

There is a characterization theorem for subobject fibrations which licenses using their internal language as the syntax for categories with finite limits.

Theorem 78 (Jacobs, 1999, 4.9.4). *Let $\pi : \mathbf{P} \rightarrow \mathbf{T}$ be a fibered preorder with finite products and equality. Then π is equivalent to a subobject fibration $\mathbf{SubT} \rightarrow \mathbf{T}$ if and only if it has extensionality*

$$\Gamma \mid (f =_B g) \vdash f = g : B$$

full subset types

$$\frac{\Gamma, x : \sigma \mid \Theta, \varphi \vdash \psi}{\Gamma, y : \{x : \sigma \mid \varphi\} \mid \Theta[o(y)/x] \vdash \psi[o(y)/x]}$$

and unique choice.

$$\frac{i : I, j_1, j_2 : J \mid R(i, j_1), R(i, j_2) \vdash j_1 = j_2}{i : I, j : J \vdash (\exists j. R \text{ prop}), (\exists j. R \sim R)}$$

The structural rules are summarized by the fact that each level forms a category.

$$\begin{array}{ccc} \frac{}{x : A \vdash x : A} & \frac{\Gamma \vdash a : A \quad x : A \vdash b : B}{\Gamma \vdash b(a) : B} & \frac{\Gamma \vdash a_1 = a_2 : A \quad x : A \vdash b : B}{\Gamma \vdash b(a_1) = b(a_2) : B} \\ \\ \frac{}{\Gamma \mid \varphi \vdash \varphi} & \frac{\Gamma \mid \varphi \vdash \theta \quad \Gamma \mid \theta \vdash \psi}{\Gamma \mid \varphi \vdash \psi} & \frac{\Gamma \vdash a : A \quad \Gamma, x : A \mid \theta \vdash \varphi}{\Gamma \mid \theta[a/x] \vdash \varphi[a/x]} \end{array}$$

units

$$\frac{}{1 \text{ type}} \quad \frac{}{\Gamma \vdash * : 1} \quad \frac{\Gamma \vdash x : 1}{\Gamma \vdash x = * : 1}$$

$$\frac{}{\Gamma \vdash \top \text{ prop}} \quad \frac{}{\Gamma \mid \varphi \vdash \top}$$

products

$$\frac{A \text{ type} \quad B \text{ type}}{A \times B \text{ type}} \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B} \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_1(p) : A} \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_2(p) : B}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \pi_1 \langle a, b \rangle = a : A} \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \pi_2 \langle a, b \rangle = b : B}$$

$$\frac{\Gamma \vdash \varphi \text{ prop} \quad \Gamma \vdash \psi \text{ prop}}{\Gamma \vdash \varphi \wedge \psi \text{ prop}} \quad \frac{\Gamma \mid \theta \vdash \varphi \quad \Gamma \mid \theta \vdash \psi}{\Gamma \mid \theta \vdash \varphi \wedge \psi} \quad \frac{\Gamma \mid \theta \vdash \varphi \wedge \psi}{\Gamma \mid \theta \vdash \varphi} \quad \frac{\Gamma \mid \theta \vdash \varphi \wedge \psi}{\Gamma \mid \theta \vdash \psi}$$

functions

$$\frac{A \text{ type} \quad B \text{ type}}{A \rightarrow B \text{ type}} \quad \frac{\Gamma, x : A \vdash f : B}{\Gamma \vdash \lambda x. f : A \rightarrow B} \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash f : A \rightarrow B}{\Gamma \vdash f a : B}$$

$\frac{\Gamma, x : A \vdash f : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda x.f) a = f[a/x] : B} \quad \frac{\Gamma \vdash f : A \rightarrow B}{\Gamma \vdash \lambda x.(f x) = f : A \rightarrow B}$	
$\frac{b : B \vdash \psi \text{ prop} \quad c : C \vdash \theta \text{ prop}}{\lambda b.c : [B \setminus C] \vdash \psi \Rightarrow \theta \text{ prop}} \quad \frac{a : A, b : B \vdash f : C \quad a : A, b : B \mid \varphi, \psi \vdash \theta[f]}{a : A \mid \varphi \vdash (\psi \Rightarrow \theta)[\lambda b.f]}$	
equality	
$\frac{\Gamma \vdash a_1 : A \quad \Gamma \vdash a_2 : A}{\Gamma \vdash (a_1 =_A a_2) \text{ prop}} \quad \frac{\Gamma, x : A \mid \theta \vdash f[x/y] =_B g[x/y]}{\Gamma, x : A, y : A \mid \theta, (x =_A y) \vdash f =_B g} \quad \frac{}{\Gamma \mid (f =_B g) \vdash f = g : B}$	
subsets	
$\frac{x : A \vdash \varphi \text{ prop}}{\{x : A \mid \varphi\} \text{ type}} \quad \frac{\Gamma \vdash a : A \quad \Gamma \mid \top \vdash \varphi[a/x]}{\Gamma \vdash i(a) : \{x : A \mid \varphi\}} \quad \frac{\Gamma \vdash a : \{x : A \mid \varphi\}}{\Gamma \vdash o(a) : A}$	
$\frac{\Gamma, x : A \mid \theta, \varphi \vdash \psi}{\Gamma, y : \{x : A \mid \varphi\} \mid \theta[o(y)/x] \vdash \psi[o(y)/x]}$	

B Free extension of theories

We will need, in §8, to say that an encoding of a data type is *canonical*. Canonicity will come from initiality, so we record the relevant machinery here.

Every finite limit category is a model of a meta-theory $\mathbb{T}_\wedge$ in $\mathbf{Set}$, presented by the judgements T_0 type, T_1 type, source and target, identities, composition, unitality, associativity, a terminal object and pullbacks; adding types $[A \rightarrow B]$ with evaluation and currying gives the meta-theory $\mathbb{T}_{\wedge \rightarrow}$ of cartesian closed categories. The full presentations are in Appendix C.

Proposition 79. *$\mathbf{FinLim} \simeq \mathbf{Cat}(\mathbb{T}_\wedge, \mathbf{Set})$, and finite-limit-preserving functors correspond to morphisms of models.*

There are analogous meta-theories for each fragment of higher-order predicate logic, connected by inclusions; and the *free model* with respect to such an inclusion is the left Kan extension along it [Barr and Wells, 2013, Section 4.4].

Proposition 80. *Given an equational theory $T : \mathbb{T}_\wedge \rightarrow \mathbf{Set}$, the free regular theory is given by the coend*

$$i_{\exists!}(T)(T) = \vec{\Sigma} S : \mathbb{T}_\wedge. T(S) \times \mathbb{T}_{\wedge \exists}(i_{\exists}(S), T)$$

and the same holds for every inclusion in the chain

$$\mathbb{T}_\wedge \subset \mathbb{T}_{\wedge \exists} \subset \mathbb{T}_{\wedge \exists \vee} \subset \mathbb{T}_{\wedge \exists \vee \vee} \subset \mathbb{T}_{\wedge \exists \vee \vee \vee} \subset \mathbb{T}_{\wedge \exists \vee \vee \vee \wedge}$$

and their closed variants.

A term of T in the free theory is a pair of a term $s : S$ in the original theory and an inference rule γ from S to T , quotiented by $\langle f(s), \gamma \rangle \equiv \langle s, \gamma(f) \rangle$. These adjunctions induce monads $(-)_\exists$ and so on. We will define bisimilarity in the free infinitary theory $\mathbb{T}_{\wedge \exists \vee \vee \vee \wedge}$, and in §8 we will use initiality in $\mathbb{T}_\wedge$ to pin down data-type encodings.

C Meta-theories

Definition 81. *The meta-theory of finite limits $\mathbb{T}_\wedge$ is a finite limit category presented as follows.*

$$\begin{array}{lcl} & \vdash & T_0 \text{ type} \\ & \vdash & T_1 \text{ type} \\ f : T_1 & \vdash & s(f) : T_0 \\ f : T_1 & \vdash & t(f) : T_0 \end{array}$$

We use $f : T(A, B)$ as shorthand for $A, B : T_0, f : T_1, s(f) = A, t(f) = B$.

$$\begin{array}{lcl} A : T_0 & \vdash & \text{id}_A : T(A, A) \\ f : T(A, B), g : T(B, C) & \vdash & g \circ f : T(s(f), t(g)) \\ f : T(A, B) & \vdash & \text{id}_A \circ f = f, \\ & & f \circ \text{id}_B = f \\ f : T(A, B), g : T(B, C), h : T(C, D) & \vdash & (h \circ g) \circ f = h \circ (g \circ f) \end{array}$$

This defines a category T ; we now give it a terminal object and pullbacks.

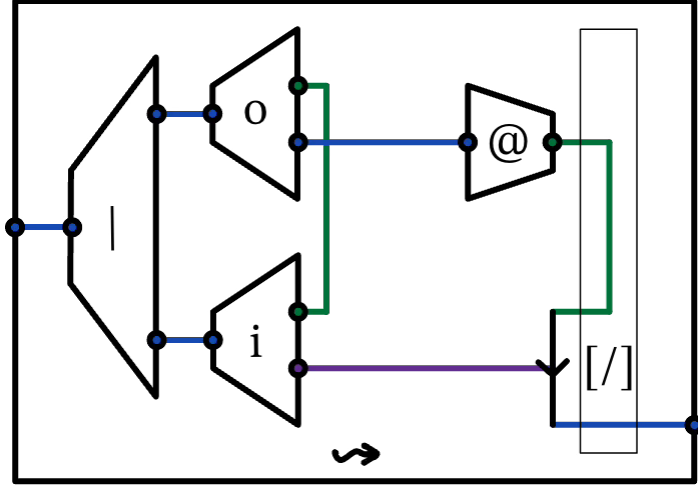
$$\begin{array}{lcl} & \vdash & 1 : T_0 \\ A : T_0 & \vdash & A_! : T(A, 1) \\ f : T(A, 1) & \vdash & f = A_! \\ \\ p : T(A, C), q : T(B, C) & \vdash & p \times_C q : T_0, \\ & & \pi_1 : T(p \times_C q, A), \\ & & \pi_2 : T(p \times_C q, B), \\ & & p \circ \pi_1 = q \circ \pi_2 \\ x : T(Z, A), y : T(Z, B) \mid p \circ x = q \circ y & \vdash & \langle x, y \rangle : T(Z, p \times_C q), \\ & & \pi_1 \circ \langle x, y \rangle = x, \\ & & \pi_2 \circ \langle x, y \rangle = y \\ ..., h : T(Z, p \times_C q) \mid \pi_1 \circ h = x, \pi_2 \circ h = y & \vdash & h = \langle x, y \rangle \end{array}$$

Definition 82. *The meta-theory of cartesian closed categories $\mathbb{T}_\wedge^\rightarrow$ is $\mathbb{T}_\wedge$ plus the following.*

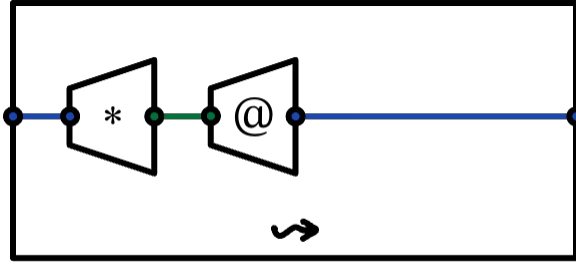
$$\begin{array}{lcl} A, B : T_0 & \vdash & [A \rightarrow B] : T_0, \\ & & \varepsilon_B^A : T([A \rightarrow B] \times A, B) \\ f : T(\Gamma \times A, B) & \vdash & \lambda a. f : T(\Gamma, [A \rightarrow B]) \\ ..., t : A & \vdash & \varepsilon_B^A(\lambda a. f, t) = f(t) \end{array}$$

D String diagrams

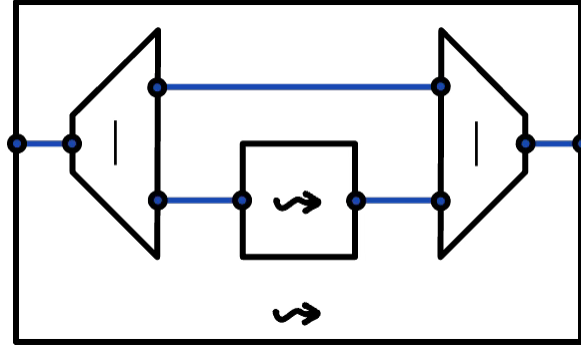
rho calculus



$$out_k(n, \vec{q}) | in_k(n, \lambda \vec{x}. p) \rightsquigarrow p[@\vec{q}/\vec{x}]$$

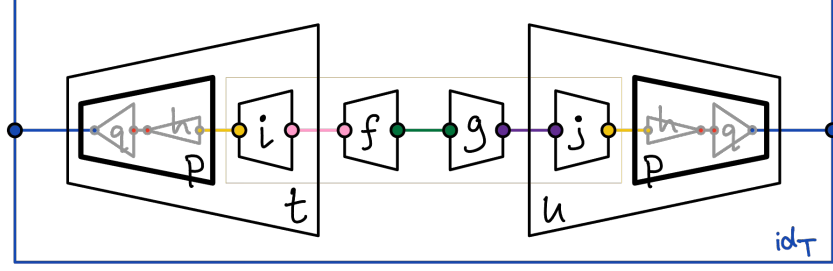


$$*@p \rightsquigarrow p$$

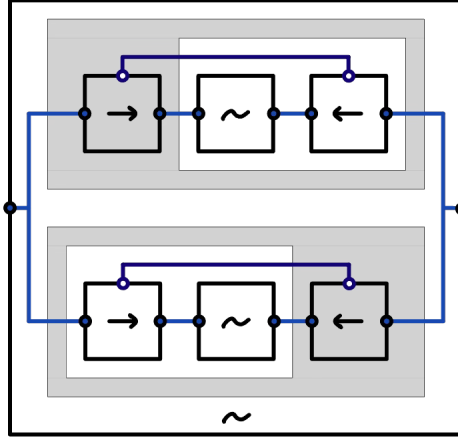
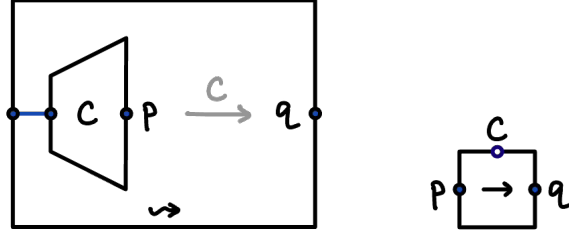


$$l \rightsquigarrow r \Rightarrow p|l \rightsquigarrow p|r$$

relative pushouts, contexts, bisimulation



The relative pushout of $t[f] = u[g]$ is $(p, (i, j))$, which factors uniquely through every q .



$$\begin{aligned}
 & p_0 \sim q_0 \\
 & \forall C, p_1. (p_0 \xrightarrow{C} p_1) \Rightarrow \exists q_1. q_0 \xrightarrow{C} q_1 \wedge p_1 \sim q_1 \\
 & \forall C, q_1. (q_0 \xrightarrow{C} q_1) \Rightarrow \exists p_1. p_0 \xrightarrow{C} p_1 \wedge p_1 \sim q_1
 \end{aligned}$$

To read each “grey bottle-opener” shape: $\forall x. P(x) \Rightarrow Q(x)$ is equivalent to $\neg[\exists x. P(x) \wedge \neg[Q(x)]]$ — the outer negation is grey, and the inner negation flips back to white.